

Project Report:

Deep Residual Learning for Image Recognition

Table of Contents

Acknowledgement.....	4
Abstract.....	5
Acronyms / Abbreviations.....	6
Introduction.....	7
Convolutional Neural Networks (CNNs).....	8
VGG Architecture.....	10
Residual Block.....	11
ResNet Architecture.....	12
Bottleneck Design.....	14
Projection and Shortcut Connections.....	15
Gradient Descent: Vanishing and Exploding Gradients.....	16
Stochastic Gradient Descent (SGD) and Mini-batch Training.....	16
End-to-End Training.....	17
Objectives.....	18
Methodology.....	19
Dataset Preparation and Preprocessing.....	19
Network Architecture Design.....	20
Training Configuration and Optimization.....	21
Experimental Tasks.....	22
Hyperparameter Optimization.....	23
Cross-Validation.....	24
Bootstrap Evaluation.....	24
Data.....	25
Experiments and Results.....	26
Discussion.....	47
Performance Analysis of Residual Architecture and Plain Architecture.....	47
Architecture Selection.....	48
Challenges.....	49
Limitations.....	49
Future Enhancements.....	50
Conclusion.....	51
References.....	52
Appendices.....	53
Author Information.....	63

List of Figures

Figure 1: Convolutional Neural Network (CNN) architecture.....	8
Figure 2: Basic VGG (Visual Geometry Group) architecture.....	9
Figure 3: Residual block of ResNet architecture.....	10
Figure 4: Comparison between VGG-19, Plain network, and Residual network.....	12
Figure 5: Residual block with and without the Bottleneck design.....	13
Figure 6: ResNet-20 Baseline loss and accuracy visualization.....	26
Figure 7: Validation loss and accuracy comparison for ResNet-20 and PlainNet-20.....	27
Figure 8: Validation loss and accuracy comparison for different depths of the ResNet model.....	28
Figure 9: Training and validation loss and accuracy for architecture using multi-skip method.....	29
Figure 10: Validation loss and accuracy for architecture of standard and multi-layer skip.....	30
Figure 11: Validation loss and accuracy for two projection methods.....	31
Figure 12: Visualization of degradation problem resolution with Residual Learning.....	32
Figure 13: Validation loss and accuracy for different ResNet architectures (all tasks summary).....	34
Figure 14: Hyperparameter Grid Search heatmap.....	35
Figure 15: Validation loss and accuracy across different Learning Rates.....	36
Figure 16: Cross Validation Fold accuracy and distribution.....	37
Figure 17: Bootstrap accuracy distribution and accuracy Violin Plot.....	38
Figure 18: Loss and Accuracy compared across training and validation sets.....	39
Figure 19: Test accuracy comparisons for all the tasks.....	40
Figure 20: Validation accuracy comparison of top 3 task models.....	40
Figure 21: Impact of network depth on the test accuracy.....	41
Figure 22: Learning rate impact on the test accuracy.....	41
Figure 23: Comparison of test accuracy for different projection methods.....	42
Figure 24: Comparison of test accuracy for different skip connection types.....	42
Figure 25: Bootstrap distribution showing accuracy across frequencies.....	43
Figure 26: Cross validation for number of folds and corresponding accuracy.....	43
Figure 27: Fundamental presentation slides.....	55
Figure 28: Output visualization for Task 1.....	56
Figure 29: Output visualization for Task 2.....	56
Figure 30: Output visualization for Task 3.....	56
Figure 31: Output visualization for Task 4.....	57
Figure 32: Output visualization for Task 5.....	57
Figure 33: Output visualization for Task 6.....	58
Figure 34: Output visualization for summary of all the tasks.....	58
Figure 35: Output visualization for Experiment 1.....	59
Figure 36: Output visualization for Experiment 2.....	59
Figure 37: Output visualization for Experiment 3.....	59
Figure 38: Output visualization for Experiment 4.....	60
Figure 39: Output visualization for Experiment 5.....	60
Figure 40: Output visualization for summary of results.....	60
Figure 41: Output table to summarize results of all tasks.....	61
Figure 42: Output table to summarize results of all experiments.....	61

List of Tables

Table 1: Training configuration and organization table.....	21
Table 2: Summary of the tasks performed along with their architecture, depth, and epochs.....	22
Table 3: Summary of the experiments performed along with their architecture, depth, and epochs.....	23
Table 4: Summarized results of different tasks.....	45
Table 5: Summarized results of different experiments.....	46

Abstract

For this project, a study of residual learning architectures for image classification was performed using the CIFAR-10 dataset, which involved analysing the degradation problem in deep neural networks and the effectiveness of skip connections in resolving optimization challenges. Through six structured tasks for core design changes and five experiments, the study compared residual networks against plain ones across depths ranging from 20 to 56 layers, while evaluating design choices including projection methods and multi-layer skip connections. The obtained results showed severe performance degradation in plain networks, with 56-layer models underperforming 20-layer versions by 6.01 percentage points, while residual networks showed consistent improvement with depth, achieving 90.65% test accuracy at 56 layers. Rigorous statistical validation through five-fold cross-validation yielded mean accuracy of 86.57% with 0.36% standard deviation, while bootstrap analysis provided a 95% confidence interval of [89.75%, 90.80%]. Hyperparameter optimization identified optimal configurations i.e., an optimized ResNet-32 achieving 91.16% test accuracy. The final results evidently validate the residual learning principles for architectural design decisions in deep Convolutional Neural Network (CNN).

Keywords: Residual Learning, CIFAR-10, Degradation Problem, Skip Connections and Convolutional Neural Network (CNN).

Acronyms / Abbreviations

ResNet: Residual Network

CNN: Convolutional Neural Network

VGG: Visual Geometry Group

SGD: Stochastic Gradient Descent

ReLU: Rectified Linear Unit

CIFAR-10: Canadian Institute For Advanced Research (10 classes dataset)

PlainNet: Plain Network

CV: Cross-Validation

LR: Learning Rate

WD: Weight Decay

COCO: Common Objects in Context

SVM: Support Vector Machine

Introduction

In the paper "Deep Residual Learning for Image Recognition" (He, Zhang, Ren, & Sun, 2016), the authors address a fundamental problem in deep learning known as the degradation problem. While deeper neural networks should theoretically be able to learn at least as well as shallower ones by simply learning identity mappings in the additional layers, experiments showed that adding more layers beyond a certain depth actually degraded performance on both training and test sets. This degradation was not caused by overfitting, as one might initially suspect, but rather by optimization difficulties that prevented the network from converging to good solutions. The authors introduced residual learning through skip connections, which fundamentally changed how deep networks learn by allowing them to learn residual mappings instead of direct mappings. This innovation enabled the training of networks with unprecedented depth, reaching up to 152 layers while achieving state-of-the-art performance on ImageNet classification with a top-5 error rate of 3.57 percent. The architecture also demonstrated strong performance on COCO (Common Objects in Context) object detection and other computer vision tasks, establishing residual connections as a foundational principle in modern deep learning.

The primary motivation of this project is to systematically validate the principles of residual learning on the CIFAR-10 dataset through an experimental approach. While the original ResNet paper demonstrated success on large-scale ImageNet images (224×224), this implementation adapts the architecture for smaller 32×32 CIFAR-10 images to investigate whether residual learning benefits generalize across different scales and problem complexities. This project work addresses several key research questions. First, it quantitatively demonstrates the degradation problem through controlled experiments comparing plain and residual networks at depths of 20, 32, 44, and 56 layers. Second, it examines design choices within the residual learning framework, including projection methods and multi-layer skip connections that bypass three convolutional layers instead of two.

The implementation employs advanced statistical methodologies for reliable evaluation. K-fold cross-validation assesses model stability across data splits, while bootstrap evaluation provides confidence intervals for performance estimates. Proper hyperparameter optimization through grid search explores how learning rate, weight decay, and momentum affect convergence and generalization. The framework tracks training efficiency metrics including convergence speed, accuracy gaps, and loss trajectories, providing insights into optimization dynamics. This systematic approach establishes a methodological template for rigorous deep learning research, demonstrating how empirical investigations should produce reliable, generalizable conclusions while ensuring transparency and reproducibility through extensive documentation, visualizations, and statistical analysis. Some of the important concepts, architectures, and terminologies used for this project have been explained below:-

Convolutional Neural Networks (CNNs)

Convolutional Neural Networks are specialized neural networks designed for processing grid-like data such as images. Unlike fully connected networks where every neuron connects to all neurons in the previous layer, CNNs use convolution operations that preserve spatial relationships in the data. A convolution operation involves sliding a small filter or kernel across the input and computing dot products at each position. Mathematically, the convolution of input I with kernel K at position i,j can be expressed as a double summation over positions m and n of the product I and K at corresponding shifted positions. This operation extracts local features while sharing parameters across the entire input, which significantly reduces the number of parameters compared to fully connected layers.

Key components of CNNs include convolutional layers that apply learnable filters to extract features like edges, textures, and patterns. Each filter specializes in detecting particular features, and multiple filters are used in each layer to capture different aspects of the input. Pooling layers reduce spatial dimensions while retaining important information, with max pooling selecting the maximum value in each region and average pooling computing the mean. Activation functions introduce non-linearity,

with ReLU being the most common choice in modern CNNs because it computes efficiently as the maximum of zero and the input value, and it helps mitigate the vanishing gradient problem. Fully connected layers appear near the end of the network to combine features for final classification or regression tasks. The Convolutional Neural Network has been visualized in figure below:-

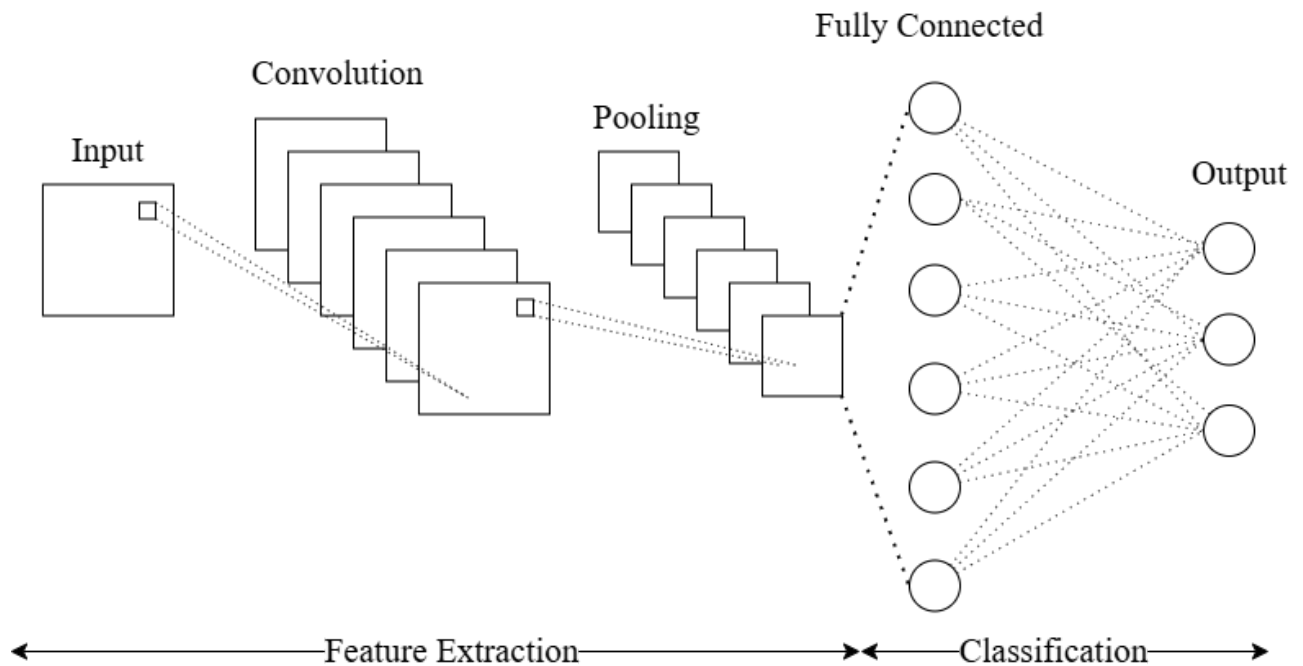


Figure 1: Convolutional Neural Network (CNN) architecture

The main advantages of CNNs stem from their architectural principles. Parameter sharing means the same filter is applied across the entire image, drastically reducing the number of parameters compared to fully connected networks. For example, a single 3x3 filter applied to a 224x224 image requires only 9 parameters and a bias, whereas a fully connected layer would require over 50,000 parameters for the same input. Spatial hierarchy allows CNNs to build increasingly abstract representations, with early layers detecting simple features like edges and corners, middle layers combining these into textures and patterns, and deeper layers recognizing object parts and complete objects. Translation invariance means that features detected in one part of the image can be recognized anywhere, making CNNs robust to variations in object position.

VGG Architecture

VGG (Visual Geometry Group) networks show that network depth is an important factor for achieving good performance in image recognition tasks. The key innovation was using very small 3x3 convolutional filters stacked in increasing depth rather than larger filters. This design choice was motivated by the observation that two 3x3 convolutional layers have the same effective receptive field as one 5x5 layer, but with fewer parameters and more non-linearity. Specifically, two stacked 3x3 layers have approximately the product of 18 and C squared parameters where C is the number of channels, versus the product of 25 and C squared for a single 5x5 layer. The additional non-linearity from having two activation functions instead of one allows the network to learn more complex decision boundaries. The basic VGG architecture has been shown in the figure below:-

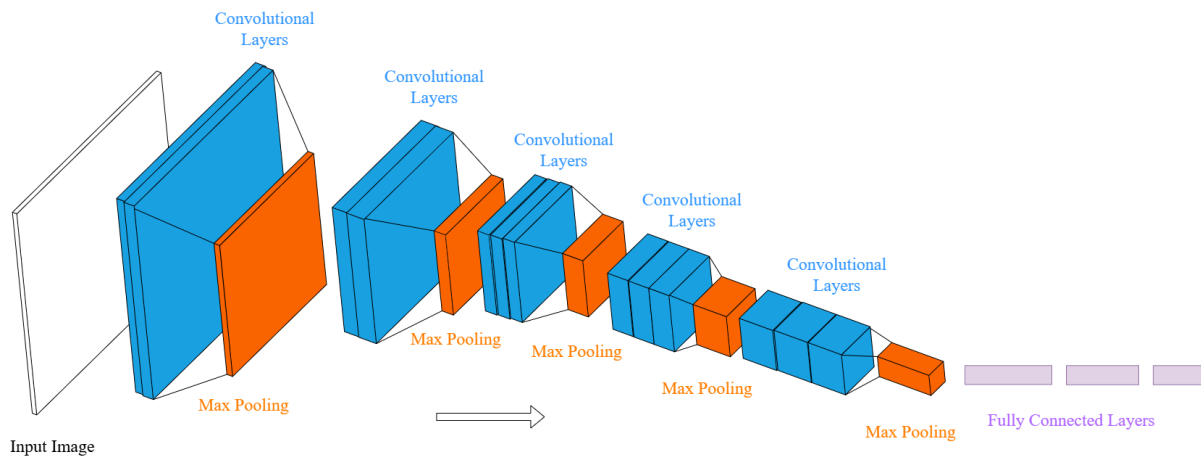


Figure 2: Basic VGG (Visual Geometry Group) architecture

The VGG-19 architecture consists of 16 convolutional layers organized in 5 blocks followed by 3 fully connected layers, totaling 19 weight layers. The architecture follows a simple and uniform pattern that begins with the input image of size 224x224 pixels with 3 color channels. The first block contains two convolutional layers with 3x3 filters and 64 output channels, followed by max pooling that reduces the spatial dimensions by half. The second block follows the same pattern with 128 channels. The third block contains four convolutional layers with 256 channels. The fourth and fifth blocks each contain four convolutional layers with 512 channels. After the convolutional blocks, the network has three fully connected layers with 4096, 4096, and 1000 neurons respectively, where the final layer produces class probabilities for ImageNet's 1000 categories.

Residual Block

The residual block is the fundamental building unit of ResNet and represents a paradigm shift in how neural networks learn. Instead of learning a direct mapping H of x from input x to output, a residual block learns the residual function $F(x) = H(x) - x$. The output is then computed as $y = F(x) + x$, where $F(x)$ represents the residual mapping to be learned and the addition operation is performed element-wise. For a basic residual block with two layers,

$F(x) = W_2 * \text{activation}(W_1) * x$, where activation is typically the ReLU function. The complete operation becomes $y = \text{ReLU}(F(x) + x)$, meaning that the final ReLU is applied after adding the shortcut connection. The residual block has been shown in the figure below:-

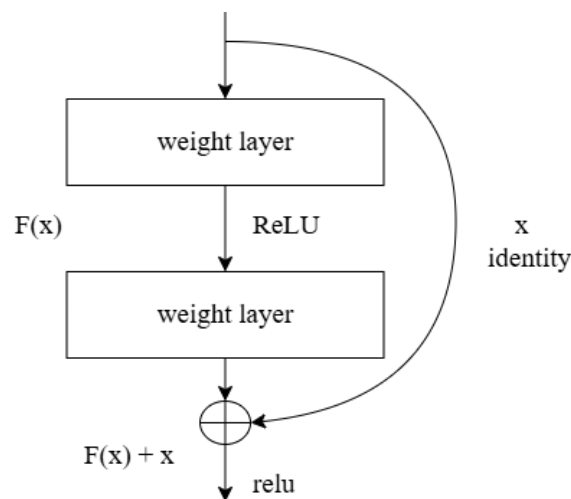


Figure 3: Residual block of ResNet architecture

The intuition behind residual learning is profound and addresses the core difficulty in training very deep networks. If the identity mapping is optimal or near-optimal for a particular layer, it is much easier for the network to push the residual $F(x)$ toward zero through weight decay and optimization than it is to fit an identity mapping using a stack of non-linear layers. When gradients flow backward during training, they can pass directly through the identity shortcut even if the gradients through the residual function F become very small. This creates what can be thought of as gradient highways that prevent the vanishing gradient problem that plagued earlier attempts at training very deep networks.

A basic residual block consists of several components arranged in a specific order. The first convolutional layer with batch normalization transforms the input, followed by a ReLU activation.

The second convolutional layer with batch normalization produces the residual output. The shortcut connection adds the original input to this residual output. Finally, a ReLU activation is applied to the sum. This structure ensures that the network can easily learn identity mappings when needed while still having the capacity to learn more complex transformations when beneficial.

ResNet Architecture

ResNet introduces a deep architecture built entirely from stacked residual blocks organized into stages. Each stage contains multiple residual blocks with the same output feature dimension, and the network transitions between stages by reducing spatial dimensions through strided convolutions while increasing the number of feature channels. This organization creates a hierarchical structure where early stages capture low-level features with high spatial resolution, and later stages capture high-level semantic features with lower spatial resolution but more feature channels.

For CIFAR-10, a ResNet-20 architecture consists of an initial 3x3 convolutional layer that transforms the 3-channel input image into 16 feature channels, followed by three stages of residual blocks. Stage 1 contains n residual blocks with 16 filters producing 32x32 feature maps. Stage 2 contains n residual blocks with 32 filters producing 16x16 feature maps, with the spatial downsampling occurring at the beginning of the stage. Stage 3 contains n residual blocks with 64 filters producing 8x8 feature maps. After the convolutional stages, global average pooling reduces each feature map to a single value, and a fully connected layer produces the final class predictions. The depth relationship follows the formula $6n + 2$ total layers, so $n = 3$ gives ResNet-20, $n = 5$ gives ResNet-32, and $n = 9$ gives ResNet-56. The general structure maintains the same principle across all depths: stacked residual blocks with periodic downsampling to reduce spatial dimensions while increasing the number of feature channels. This design allows the network to progressively transform the input from raw pixel values to high-level semantic representations suitable for classification or other tasks.

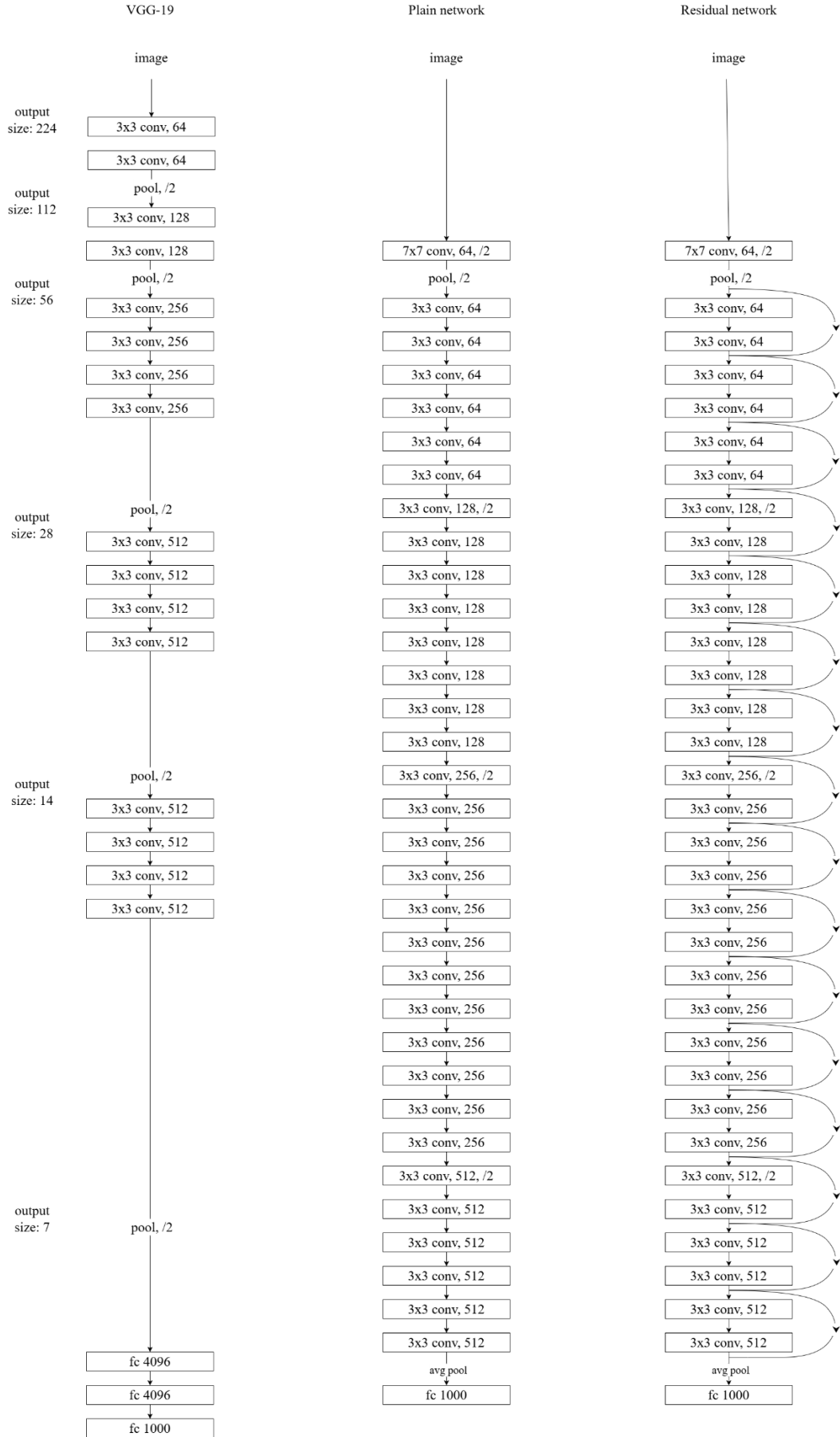


Figure 4: Comparison between VGG-19, Plain network, and Residual network

Bottleneck Design

For very deep networks with 50 or more layers, ResNet employs a bottleneck design to reduce computational complexity while maintaining representational power. Instead of two 3x3 convolutions, a bottleneck block uses three layers arranged as a 1x1 convolution to reduce dimensions, a 3x3 convolution to process features, and another 1x1 convolution to restore dimensions. Mathematically, for input x with 256 channels, $F(x) = W3 * \text{activation}(W2 * \text{activation}(W1 * x))$, where $W1$ is a 1x1 convolution reducing 256 channels to 64 channels, $W2$ is a 3x3 convolution maintaining 64 channels, and $W3$ is a 1x1 convolution expanding 64 channels back to 256 channels. The comparison has been visualized below:-

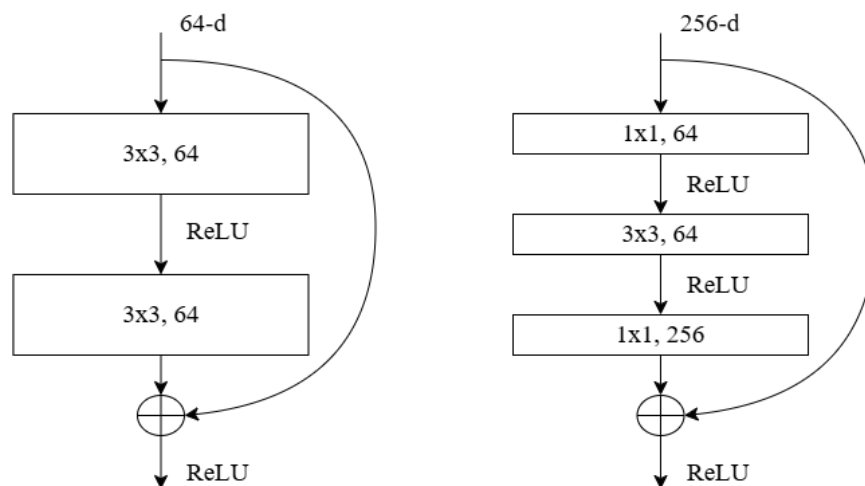


Figure 5: Residual block with and without the Bottleneck design

The bottleneck design significantly reduces the number of parameters and computational cost. For a concrete comparison, consider transforming between 256-channel feature maps. A basic block with two 3x3 convolutions requires approximately 1,179,648 parameters. In contrast, a bottleneck block requires approximately 69,632 parameters. This represents an approximate 17-fold reduction in parameters while maintaining or even improving performance. The bottleneck works because the expensive 3x3 convolution operates on a reduced number of channels, and the 1x1 convolutions efficiently compress and expand the representation. This design principle has been widely adopted in subsequent architectures because it allows building very deep networks that remain computationally tractable.

Projection and Shortcut Connections

When the dimensions of the input and output differ due to changes in spatial size or number of channels, the shortcut connection must be modified to match dimensions. The ResNet paper explores two main options for handling dimension mismatches. Projection A uses zero-padding for increased dimensions combined with strided convolution or pooling for downsampling. This approach is parameter-free, which means it does not add any trainable parameters to the network and cannot overfit. Projection B uses projection shortcuts implemented as 1×1 convolutions to match dimensions. The operation $y = F(x) + W_s * x$, where W_s is a 1×1 convolution that adjusts both spatial size through its stride parameter and number of channels through its filter count. When input and output dimensions match, the identity shortcut is always used, expressed simply as $y = F(x) + x$. This is the most common case within a stage where all blocks have the same dimensions.

The paper's experiments found that projection shortcuts provide slightly better performance, particularly when all shortcuts use projections rather than just the dimension-changing ones. However, the performance difference is small, and zero-padding shortcuts successfully resolve the degradation problem while being more parameter-efficient. The choice between these options represents a tradeoff between model capacity and parameter efficiency. Modern implementations often use projection shortcuts for dimension changes while using identity shortcuts for matching dimensions, as this provides good performance without excessive parameters.

Gradient Descent: Vanishing and Exploding Gradients

During backpropagation, gradients are computed using the chain rule, which requires multiplying derivatives from each layer as the gradient propagates backward through the network. For a network with L layers, the gradient with respect to early layer parameters involves a product of many derivatives. The problem arises when these derivative terms are consistently either much less than one or much greater than one. The vanishing gradient problem occurs when derivatives are less than one, causing gradients to exponentially decrease as we move backward through the network. This causes early layers to learn extremely slowly or not at all because their gradients become vanishingly small, often falling below the numerical precision of floating-point arithmetic. Early layers are precisely those that should learn basic features, so their failure to learn properly severely limits network performance. This problem was particularly acute with sigmoid and hyperbolic tangent activation functions, which have derivatives that are always less than one except at a single point. On the other hand, the exploding gradient problem occurs when derivatives are greater than one, causing gradients to grow exponentially. This leads to numerical instability where weight updates become so large that they cause the loss to diverge rather than converge. While gradient clipping can partially address exploding gradients by capping their magnitude, this is only treating the symptom rather than the underlying cause.

Stochastic Gradient Descent (SGD) and Mini-batch Training

Stochastic Gradient Descent updates model parameters by computing gradients on small batches of data rather than the entire dataset. Standard gradient descent computes the gradient over all training examples. While this guarantees convergence in convex problems and movement in the steepest descent direction, it is computationally prohibitive for large datasets and makes learning very slow. Mini-batch SGD addresses this by approximating the full gradient using a small batch of examples. This approach provides several advantages. It reduces computational cost per update, allows for parallel processing on GPUs, introduces beneficial noise that can help escape local minima, and enables training on datasets too large to fit in memory. The stochasticity means each update is based

on a different subset of data, which helps the optimizer explore the parameter space more thoroughly. ResNet training uses SGD with momentum to accelerate convergence and smooth out the optimization trajectory. Standard momentum maintains a velocity term that accumulates gradients over time. Momentum helps the optimizer build up speed in directions of consistent gradient, allowing it to move faster through flat regions and dampen oscillations in steep ravines.

End-to-End Training

End-to-end training means the entire network is trained jointly with a single loss function, without requiring separate training stages or hand-crafted features. For ResNet on ImageNet classification, the loss function is cross-entropy. The network learns all representations automatically through backpropagation, with low-level features such as edges and textures emerging in early layers, mid-level features such as parts and patterns in middle layers, high-level features in deep layers, and classification in the final layer. This approach contrasts sharply with traditional computer vision pipelines that required multiple manual steps. Classical systems used hand-crafted feature extraction methods like Scale-Invariant Feature Transform for detecting keypoints and computing local descriptors, Histogram of Oriented Gradients for capturing gradient orientations, or color histograms for representing appearance. These features were designed by human experts based on intuition about what properties discriminate between different objects or scenes. After feature extraction, a separate classifier such as Support Vector Machine (SVM) or random forest would be trained on these fixed features.

End-to-end training with deep residual networks eliminates all these manual steps. The network learns features directly from raw pixels, discovering representations that are optimal for the task at hand rather than relying on human-designed features. The hierarchical nature of CNNs naturally captures multi-scale information, with early layers responding to local patterns and deeper layers integrating information over larger spatial regions to recognize complete objects. The residual connections enable very deep networks that can learn increasingly abstract representations while maintaining trainability.

This end-to-end approach has proven significantly more effective than traditional pipelines, achieving superhuman performance on many visual recognition tasks.

Objectives

The objectives of this project has been discussed below:-

1. To implement baseline ResNet-20 on CIFAR-10 and compare residual networks against plain networks at identical depths to empirically demonstrate the degradation problem.
2. To train ResNets and plain networks at depths of 20, 32, 44, and 56 layers to verify that residual connections enable successful depth scaling while plain networks degrade with increased depth.
3. To evaluate architectural variants including multi-layer skip connections and projection versus zero-padding shortcuts to analyze their impact on accuracy and parameter efficiency. This is the major objective of the entire project.
4. To perform hyperparameter optimization through grid search and learning rate sensitivity analysis to identify optimal training configurations for deep residual networks.
5. To conduct statistical validation using 5-fold cross-validation and bootstrap evaluation to assess performance stability and construct confidence intervals for proper generalization estimates.
6. To train optimized ResNet-32 using best hyperparameters to achieve maximum performance and validate the effectiveness of systematic experimentation and residual learning principles.

Methodology

Dataset Preparation and Preprocessing

The CIFAR-10 dataset, a widely recognized benchmark for image classification tasks consisting of 60,000 32×32 color images distributed across 10 distinct classes, was used for this implementation. The dataset was partitioned into 50,000 training images and 10,000 test images. An 80-20 split was applied to the training set, resulting in 40,000 images for training and 10,000 images for validation purposes. Moreover, the stratified division was implemented to ensure that model performance could be evaluated on data during the training process while preserving a separate test set for final assessment. Additionally, data augmentation techniques were implemented to enhance model generalization and prevent overfitting. The training data underwent random horizontal flips with 0.5 probability, random crops of 32×32 pixels with 4-pixel padding, random rotations up to 15 degrees, and color jitter transformations affecting brightness, contrast, saturation, and hue with factors of 0.2, 0.2, 0.2, and 0.1 respectively. Then, all images were normalized by subtracting the per-channel mean computed across spatial dimensions. The validation and test sets received identical normalization but without augmentation to maintain consistency in evaluation. The 10 output classes in the CIFAR-10 dataset are as follows:-

- i. airplane
- ii. automobile
- iii. bird
- iv. cat
- v. deer
- vi. dog'
- vii. frog
- viii. horse
- ix. ship
- x. truck

Network Architecture Design

The basic architecture implemented was a residual network, following the design principles established in the original ResNet paper but scaled appropriately for 32×32 input images. The network consisted of an initial convolutional layer with 16 filters, three stages of residual blocks with 16, 32, and 64 filters respectively. Each stage contained n residual blocks, where n was determined by the target network depth according to the formula $depth = 6n + 2$. The residual block implementation incorporated two 3×3 convolutional layers with batch normalization applied after each convolution. When spatial dimensions needed to be reduced, stride-2 convolutions were employed in the first layer of designated downsampling blocks. Two projection strategies were examined for matching dimensions in skip connections. The first approach used zero-padding to increase channel dimensionality, while the second approach employed 1×1 convolutions to project the shortcut to the desired dimensions. The final classification head utilized global average pooling followed by a fully connected layer mapping to 10 classes. Multiple architectural variants were constructed to enable proper comparison. Plain networks were created by removing skip connections entirely while maintaining identical layer structures. Multi-layer skip connections were investigated through blocks that aggregated outputs across three convolutional layers before adding the residual. Network depths of 20, 32, 44, 56, and 110 layers were instantiated by varying n in the depth formula, allowing examination of scaling properties. All convolutional layers received Kaiming normal initialization, specifically designed for layers with ReLU activations. This initialization scheme sets weights according to a normal distribution with zero mean and variance equal to 2 divided by the number of input channels multiplied by kernel size. Bias terms, when present, were initialized to zero. This initialization approach was important for enabling training of deep networks by maintaining appropriate signal variance across layers.

Training Configuration and Optimization

The base training configuration employed stochastic gradient descent with momentum as the optimization algorithm. The initial learning rate was set to 0.1, with momentum of 0.9 and weight decay of 0.0001. Nesterov momentum was enabled to provide look-ahead gradient estimates. The learning rate schedule followed a multi-step decay pattern, with the rate multiplied by 0.1 at epochs 80 and 120 for the standard 160-epoch training runs. Shorter experiments used proportionally scaled milestone epochs. Training proceeded in mini-batches of 128 images. For each epoch, the model performed a forward pass computing cross-entropy loss, followed by backward propagation to calculate gradients, and finally a parameter update via the optimizer. Training accuracy was computed by comparing predicted class labels to ground truth, while validation performance was assessed after each epoch using the same metrics on the held-out validation set. The model state achieving the highest validation accuracy was preserved throughout training. This has been summarized in the table below:-

Parameter	Value(s)
Base Learning Rate: Initial learning rate for most experiments	0.1
Momentum	0.9
Weight Decay: L2 regularization factor	1×10^{-4}
Nesterov Momentum: Look-ahead gradient estimation	Enabled
Batch Size using mini-batch	128
Training Epochs	50-160
Learning Rate Milestones: Epoch numbers for LR reduction	[80, 120] or [50, 75]
Loss Function	Cross-Entropy
Weight Initialization for Conv2d layers with ReLU	Kaiming Normal

Table 1: Training configuration and organization table

Experimental Tasks

Six primary tasks were designed to investigate different aspects of residual learning. Task 1 established a baseline by training ResNet-20 with option A projection for 100 epochs. Task 2 compared residual and plain architectures by training ResNet-20 with option B projection against a PlainNet-20 for 80 epochs each, examining the fundamental benefit of skip connections. Task 3 explored depth scaling by training networks of 20, 32, and 44 layers with identical configurations, revealing how performance changed with network capacity. Task 4 examined multi-layer skip connections that bypassed three convolutional layers rather than two, trained for 100 epochs to assess whether longer skip paths offered advantages. Task 5 contrasted the two projection methods by training ResNet-32 with both option A and option B for 70 epochs, quantifying the impact of projection strategy choice. Task 6 provided the analysis of the degradation problem by training both plain and residual networks at depths 20, 32, 44, and 56 for 80 epochs, systematically documenting how each architecture responded to increased depth. All the tasks have been summed up in the table below:-

Task	Architectures Tested	Depth(s)	Epochs
Task 1 - Baseline ResNet-20: Establishing the baseline performance	ResNet-20	20	100
Task 2 - Residual vs Plain: Comparison of skip connections with plain networks	ResNet-20 and PlainNet-20	20	80
Task 3 - Depth Comparison: Evaluation of scaling with network depth	ResNet-20/32/44	20, 32, 44	80
Task 4 - Multi-Layer Skip: Evaluating 3-layer skip connections	ResNet-20 Multi-Skip	20	100
Task 5 - Projection Methods: Comparison of projection methods	ResNet-32	32	70
Task 6 - Degradation Analysis: To show that the degradation problem has been solved	PlainNet and ResNet	20, 32, 44, 56	80

Table 2: Summary of the tasks performed along with their architecture, depth, and epochs

Hyperparameter Optimization

A grid search methodology was employed to identify optimal hyperparameter configurations. A reduced subset of 5,000 training samples and 1,000 validation samples was used to enable rapid experimentation. The search space covered learning rates of 0.05, 0.1, and 0.15, weight decay values of 0.0001, 0.0005, and 0.001, and momentum coefficients of 0.85, 0.9, and 0.95. Each combination was trained for 40 epochs with batch size 128, resulting in 27 total configurations evaluated. The configuration achieving the highest validation accuracy was identified as optimal. Learning rate sensitivity was investigated through a dedicated experiment training ResNet-20 with learning rates of 0.01, 0.05, 0.1, and 0.2 for 60 epochs each. This isolated the impact of learning rate magnitude on convergence speed and final performance, with all other hyperparameters held constant. The experiments performed during hyperparameter optimization are as follows:-

Experiment Name	Method	Configurations / Samples / Folds	Duration
Experiment 1 - Grid Search: To evaluate the best validation accuracy	Exhaustive grid search	27 configurations	40 epochs each
Experiment 2 - LR Sensitivity: For convergence and final accuracy	Systematic Learning Rate variation	4 configurations	60 epochs each
Experiment 3 - Cross Validation (CV)	5-fold Cross Validation	5 folds	50 epochs/fold
Experiment 4 - Bootstrap Evaluation	Resampling with replacement	20 samples	N/A
Experiment 5 - Optimized Configuration to evaluate the most optimal performance	Best practices combined	Single execution	120 epochs

Table 3: Summary of the experiments performed along with their architecture, depth, and epochs

Cross-Validation

Five-fold cross-validation was conducted to assess model performance stability and obtain reliable accuracy estimates. The training dataset was randomly partitioned into five equal folds. For each fold, the model was trained on four folds and validated on the remaining fold, with this process repeated five times such that each fold served as validation exactly once. Each fold was trained for 50 epochs using the standard hyperparameter configuration. The best validation accuracy from each fold was recorded, and summary statistics including mean, standard deviation, minimum, and maximum were computed across folds.

Bootstrap Evaluation

Bootstrap resampling was applied to the test set to estimate confidence intervals for model performance. Twenty bootstrap samples were drawn, each containing 80% of the test set selected with replacement, allowing some images to appear multiple times while others were excluded. The trained baseline model was evaluated on each bootstrap sample, computing test accuracy for all 20 replicates. The mean and standard deviation of these accuracies provided point estimates and variability measures, while the 2.5th and 97.5th percentiles yielded 95% confidence intervals.

Data

For this project, the CIFAR-10 dataset has been used. The CIFAR-10 dataset is a widely used benchmark in computer vision for image classification tasks. It consists of 60,000 color images of size 32×32 pixels, divided into 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The dataset is split into 50,000 training images and 10,000 test images, with each class containing an equal number of images, making it a balanced dataset. The original 50,000 training images were further partitioned into an 80% training subset (40,000 images) and a 20% validation subset (10,000 images) using a random split method. This dedicated validation set was utilized for monitoring performance during training and selecting the best model checkpoint.

CIFAR-10 images capture a variety of objects in natural scenes, providing diverse examples of textures, colors, and shapes, which challenges models to learn meaningful features despite the small image size. Its compact size allows rapid experimentation with deep learning architectures, while its diversity and class balance make it an ideal benchmark for evaluating and comparing the performance of convolutional neural networks and other image classification models. The dataset has been extensively used in research, providing a standard reference point for measuring generalization, robustness, and the effectiveness of different model designs.

Experiments and Results

For this project, the implementation was organized into two complementary components, where the tasks focused on core architectural comparisons and design validation, while the experiments solely focused on the hyperparameter optimization and statistical analysis. The task component systematically evaluated the fundamental principles such as the degradation problem, depth scaling and architectural variations across the networks. The experiment component provided rigorous statistical validation through cross-validation and bootstrapping methods, to identify the optimal training configurations.

The results of different tasks and experiments performed throughout this project have been discussed in this section. The loss and accuracy curves for the baseline ResNet-32 are shown below:-

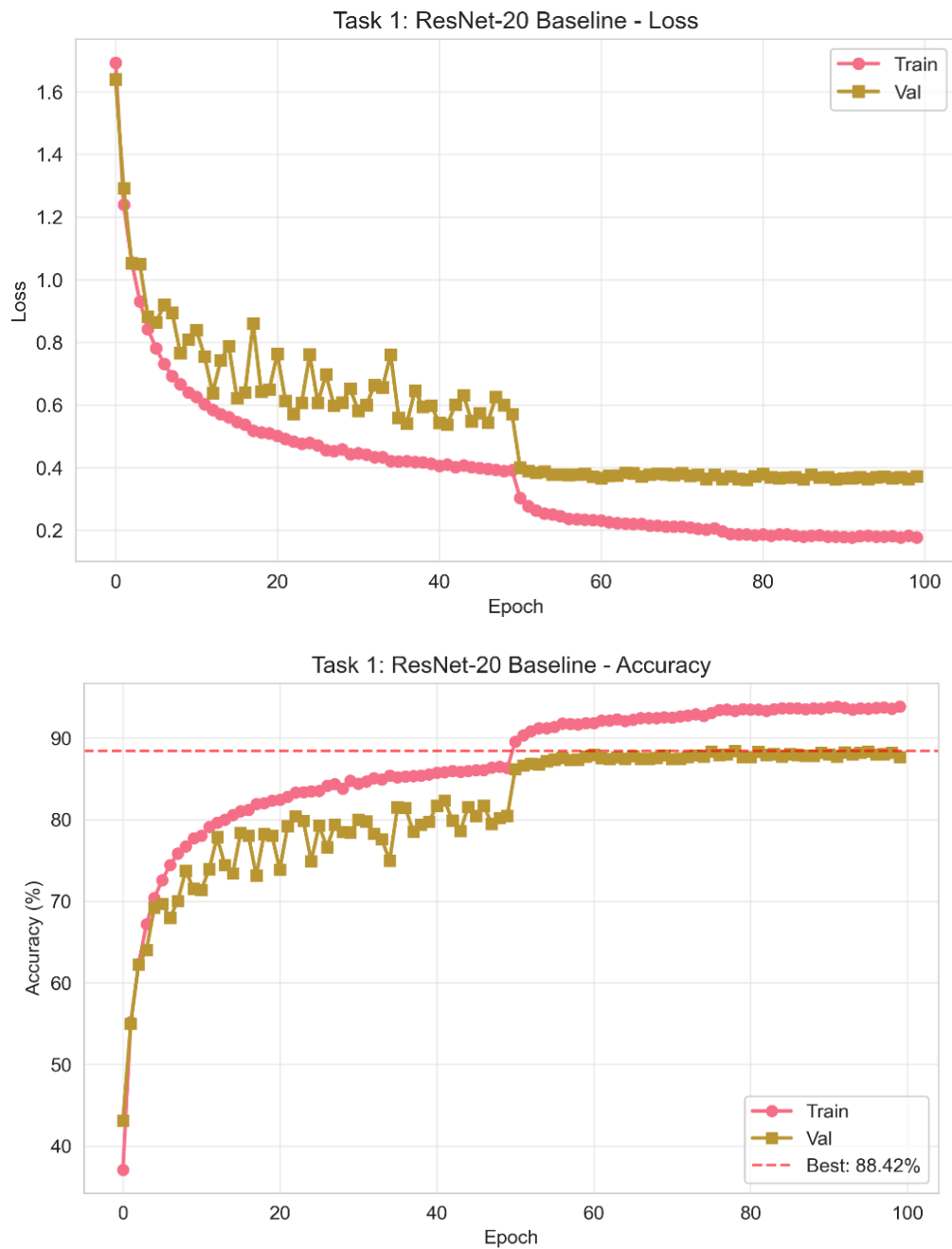


Figure 6: ResNet-20 Baseline loss and accuracy visualization

The "Task 1: ResNet-20 Baseline" charts establish the foundational performance for the study. The model achieves a validation accuracy of 88.42%. The training dynamics have a rapid initial learning phase followed by steady convergence. This baseline validates the correct implementation of the ResNet architecture and provides a reference point for any adjustments.

The curves showing the validation loss and validation accuracy across the ResNet-20 and PlainNet-20 architecture are as follows:-

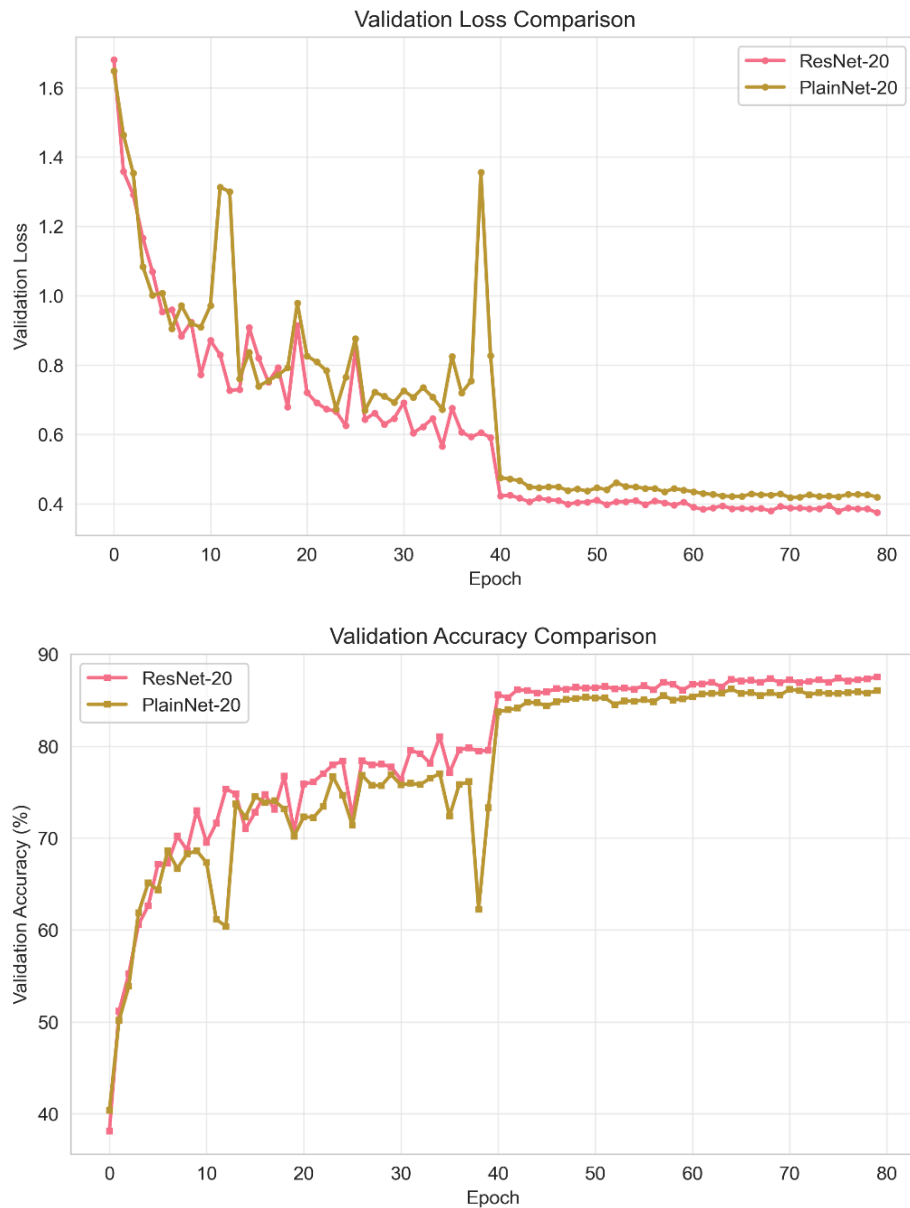


Figure 7: Validation loss and accuracy comparison for ResNet-20 and PlainNet-20

The comparison in the graphs above gives validation of the residual learning hypothesis. The ResNet-20 demonstrates a clear and consistent advantage over the PlainNet-20. The PlainNet struggles to achieve the same validation loss and lags in accuracy by several percentage points throughout training. This gap shows that even at a relatively shallow depth of 20 layers, the introduction of skip connections significantly aids optimization. This is likely caused by mitigating the early stages of the vanishing gradient problem and allowing for easier identity mapping.

The validation loss and accuracy curves comparing different architectures have been shown below:-

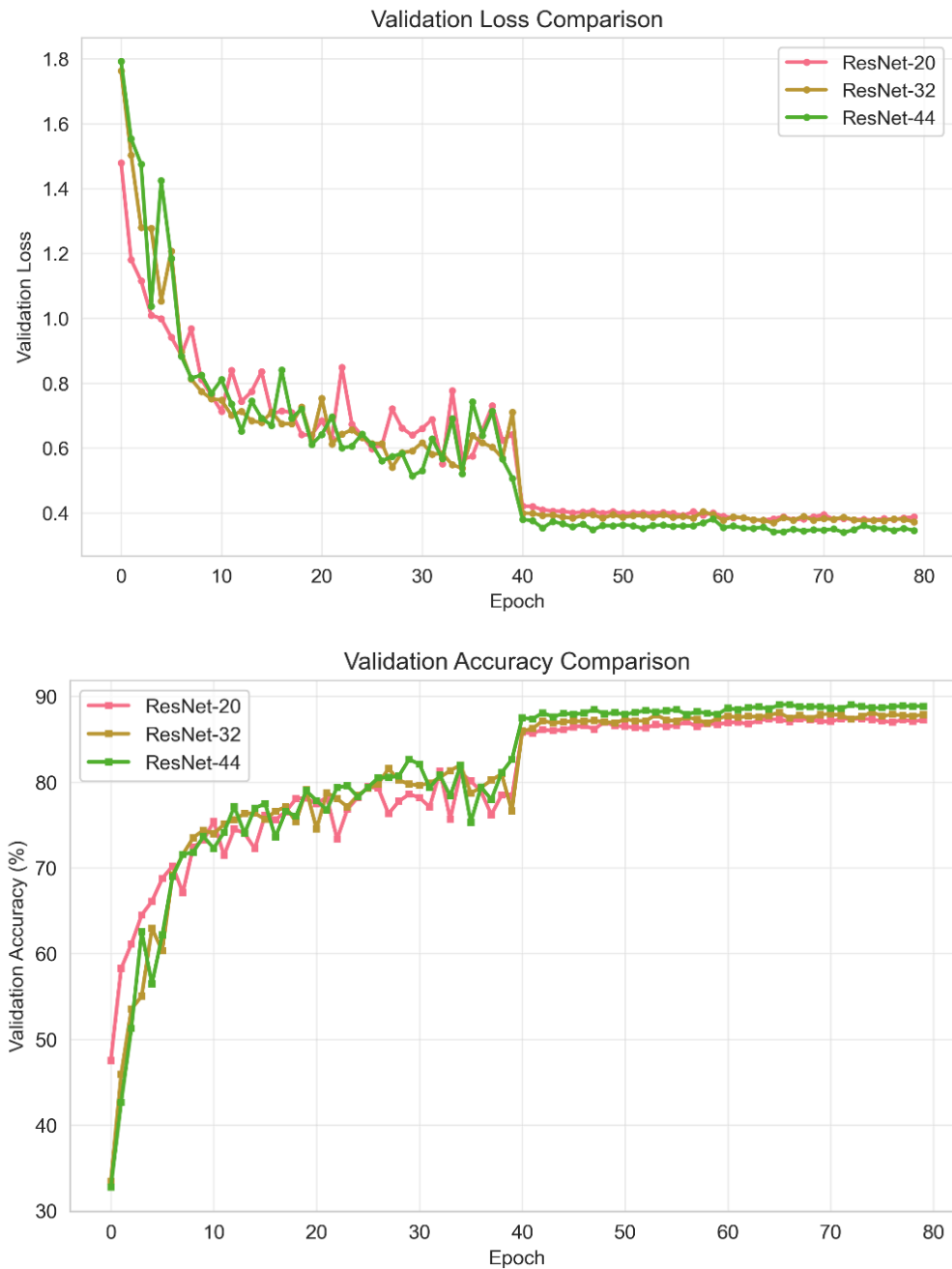


Figure 8: Validation loss and accuracy comparison for different depths of the ResNet model

The Validation Loss and Accuracy Comparison charts for Task 3 investigate scalability. There is a positive correlation between depth and performance. The ResNet-44 outperforms the ResNet-32, which beats the ResNet-20. These ResNets models successfully leverage the additional parameters to learn more complex features and push the accuracy from the approximately 88% range up towards approximately nearly 90%. This confirms that the residual architecture successfully allows for deeper networks to be trained without degradation.

The validation loss and accuracy for the ResNet-20 network using multiple skip connections have been shown below:-

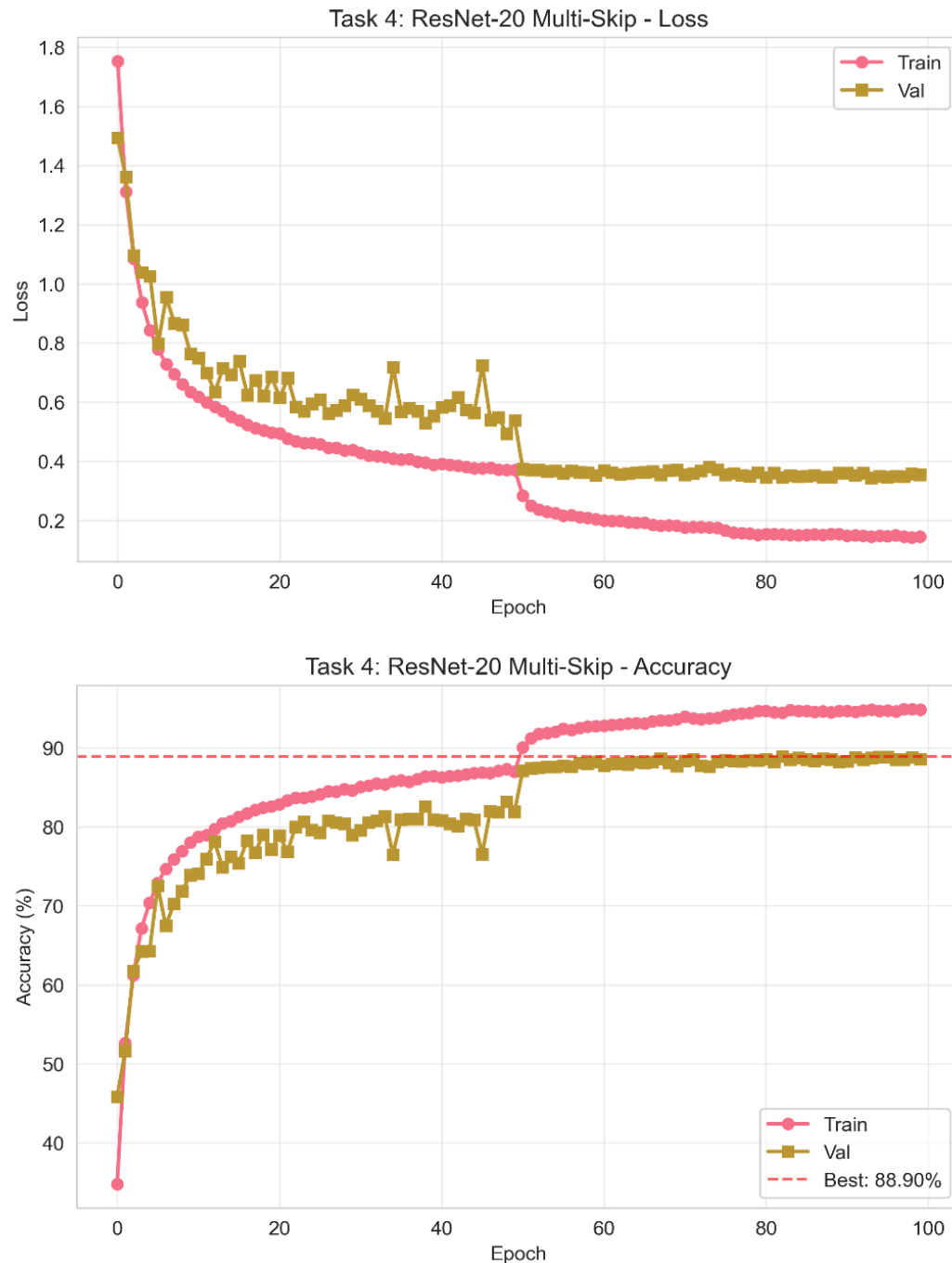


Figure 9: Training and validation loss and accuracy for architecture using multi-skip method

Task 4 covers the Multi-Layer Skip variant of the ResNet model, and the graphs show a marginal improvement over the standard skip, achieving a best accuracy of 88.90%. While the difference is subtle, the multi-layer skip appears to offer slightly more stability in the loss curve. This suggests that having even more direct pathways for gradient flow across multiple layers can give slight additional performance gains.

The validation loss and accuracy comparison for the standard skip connection and multi-layer skip has been shown below:-

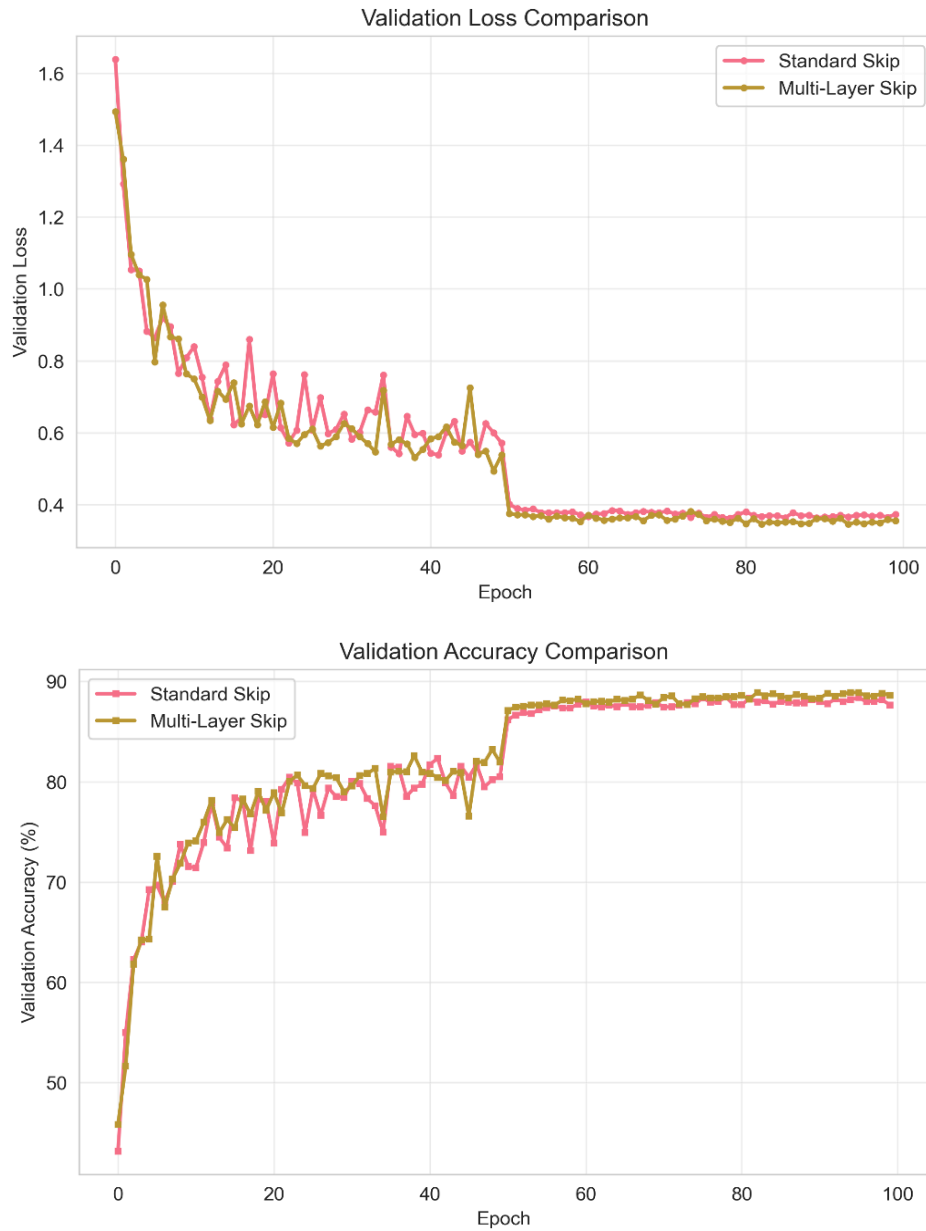


Figure 10: Validation loss and accuracy for architecture of standard and multi-layer skip

The charts above show the validation loss and validation accuracy when architectures are implemented using standard skip and when architectures have been implemented using the multi-layer skip. From both of the above figures, it is evident that there is not much difference in both the loss and accuracy for both types of networks. However, an insightful pattern visible in both of the graphs is that, when architectures have been implemented using the standard skip, the loss and accuracy start fluctuating significantly in between 10 to 40 epochs with major instabilities that can be concerning. This can be due to the effect of the residual blocks when such architecture has not been explored.

The validation loss and validation accuracy across different projections have been shown below:-

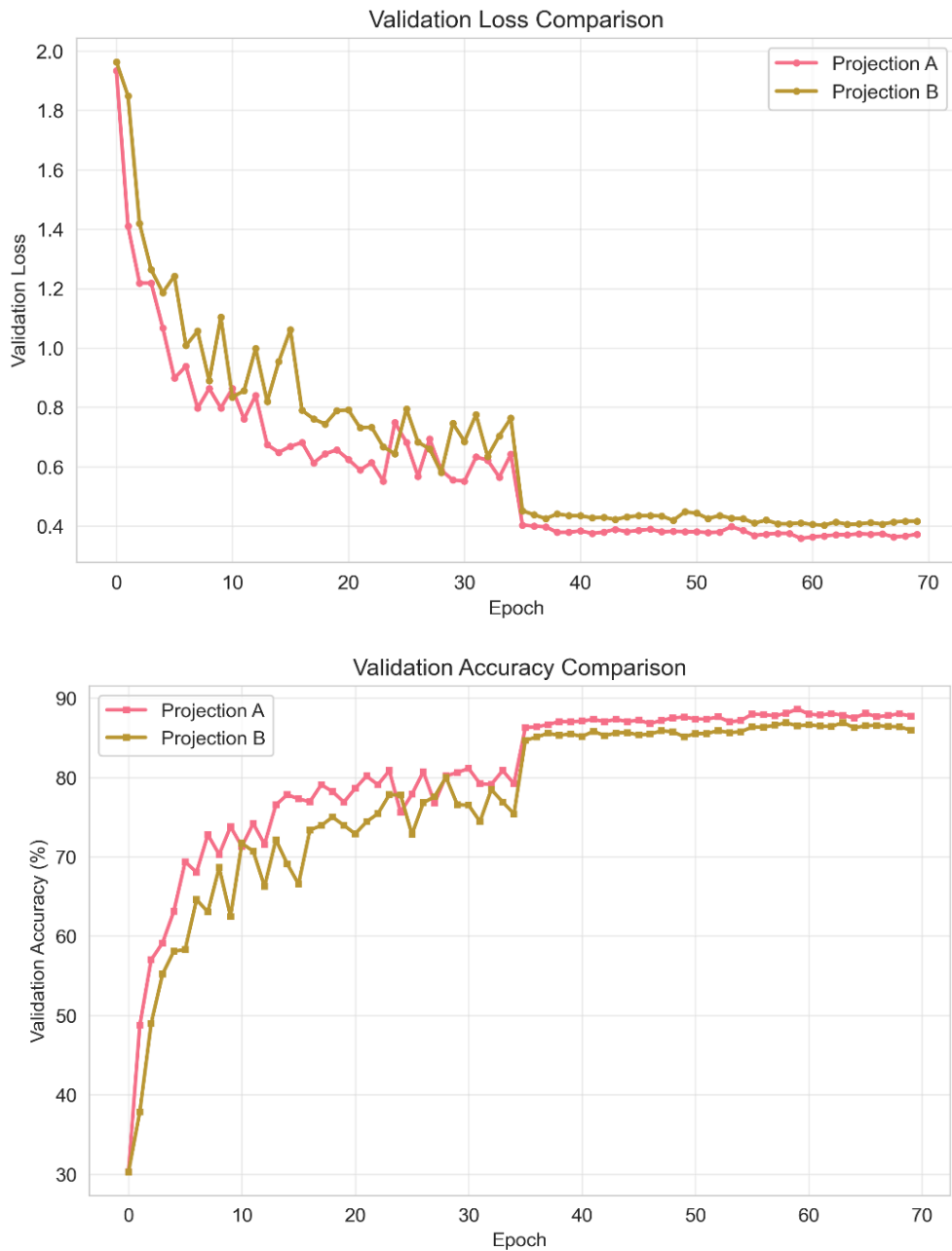


Figure 11: Validation loss and accuracy for two projection methods

For task 5, Projection A that is the projection with zero-padding clearly outperforms Projection B (1x1 convolutions). Projection A achieves lower validation loss and higher accuracy faster. This suggests that for this specific dataset and model size, the parameter-free zero-padding approach is superior. It acts as a form of regularization, whereas the learned 1x1 convolutions in Projection B might be introducing unnecessary complexity or parameters that lead the optimization process into a suboptimal local minimum.

To realize the issue this project is attempting to solve, the visualizations below show the differences in models with and without the solutions to tackle the degradation issue as follows:-

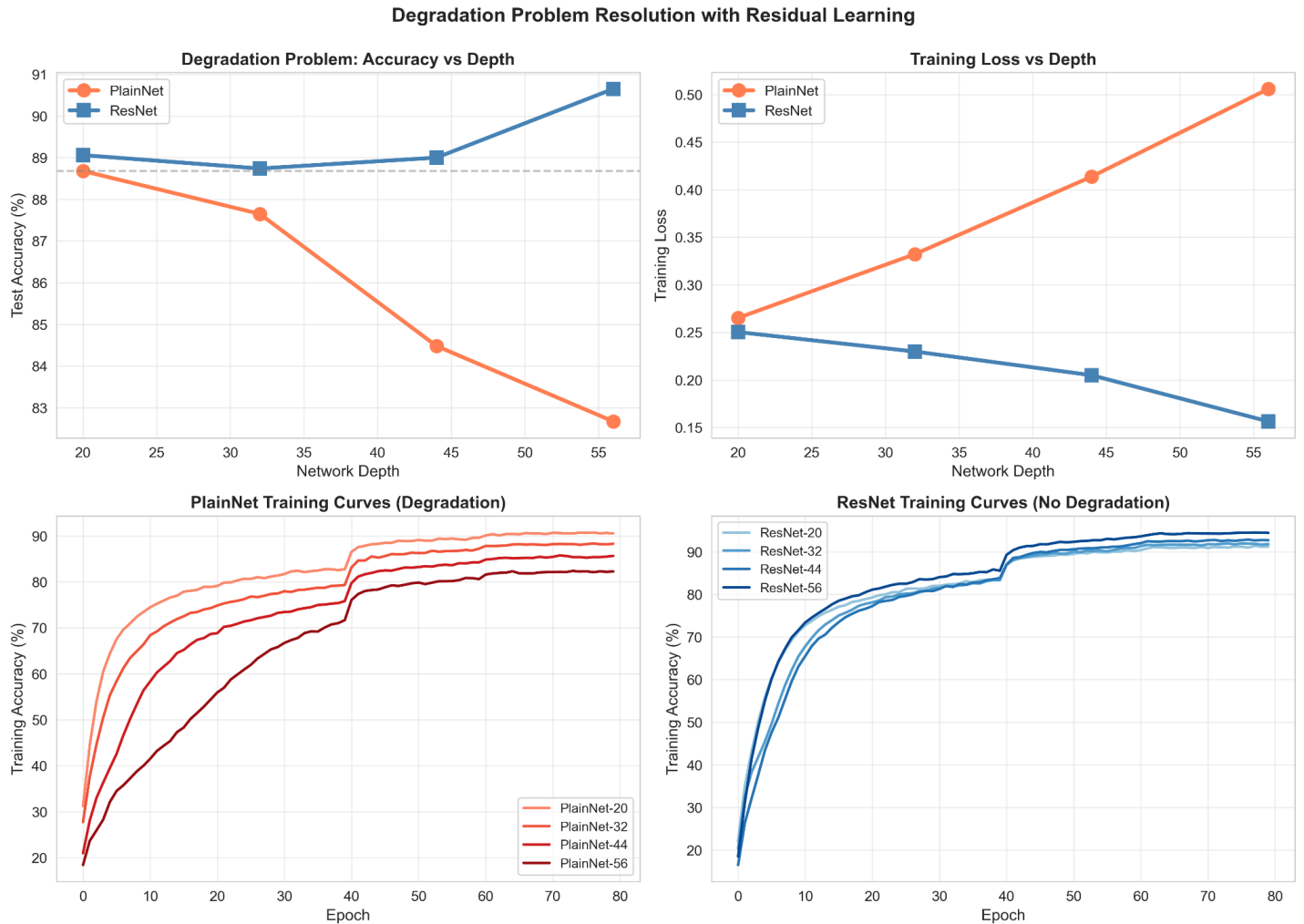


Figure 12: Visualization of degradation problem resolution with Residual Learning

These graphs illustrate the difference in performance between the ResNet and PlainNet architectures. The degradation problem's Accuracy versus Depth graph shows that PlainNet suffers from the degradation problem. As the network depth increases from 20 to 56 layers, its test accuracy steadily declines from approximately 88.7% to under 83%. In contrast, ResNet effectively uses increased depth, with accuracy improving from roughly 89.1% at 20 layers to nearly 90.7% at 56 layers. The Training Loss vs Depth chart provides evidence for this. The deeper PlainNets struggle to optimize, exhibiting higher training loss, while the deeper ResNets successfully lower their training loss. The graphs in the second row dive into the training dynamics that cause these outcomes. The PlainNet Training Curves for degradation reveal that adding layers hinders convergence, with the shallowest

model (PlainNet-20) achieving significantly higher training accuracy than the deepest model (PlainNet-56) throughout the entire training process. The "ResNet Training Curves" demonstrate the opposite trend. The deeper networks consistently achieve higher training accuracy than shallower ones. This visual evidence shows that residual connections (skip connections) successfully solve the degradation problem by enabling deeper networks to learn identity mappings and optimize more effectively, turning increased depth from a liability into an advantage.

The validation loss and validation accuracy has been compared for different architectures as follows:-

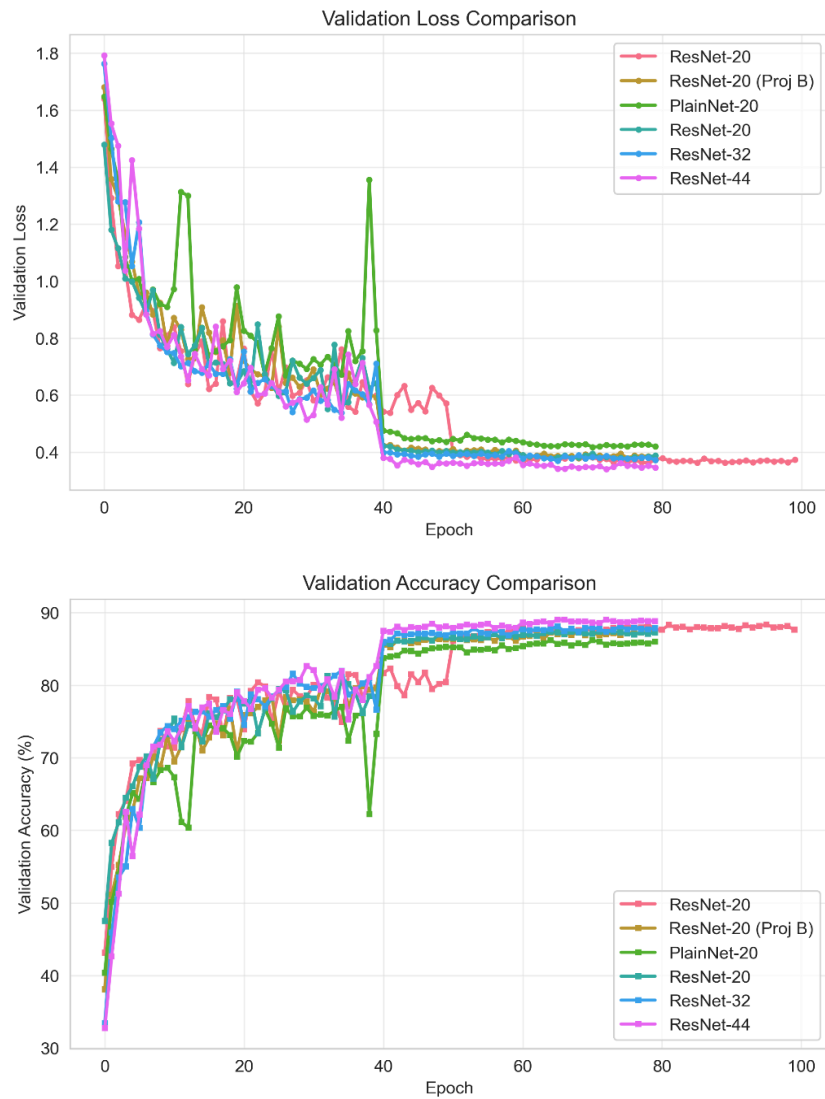


Figure 13: Validation loss and accuracy for different ResNet architectures (all tasks summary)

These summary charts provide a general overview of the performance hierarchy among the tested architectures. The PlainNet-20 consistently lags behind all residual network variants, showing higher validation loss and lower accuracy throughout the training process. This provides evidence for the fundamental advantage of residual learning even at shallower depths. For the ResNet models, a positive correlation between depth and performance is evident. ResNet-44 and ResNet-32 achieve the lowest final loss and highest validation accuracy, clearly outperforming the ResNet-20 variants. This performance validates that deeper ResNets can successfully use their increase in layers to learn more complex features without the optimization difficulties that plain networks suffer from, effectively turning increased depth into performance gain.

The heatmap showing the hyperparameter grid search across different learning rates and weight decay have been shown below:-

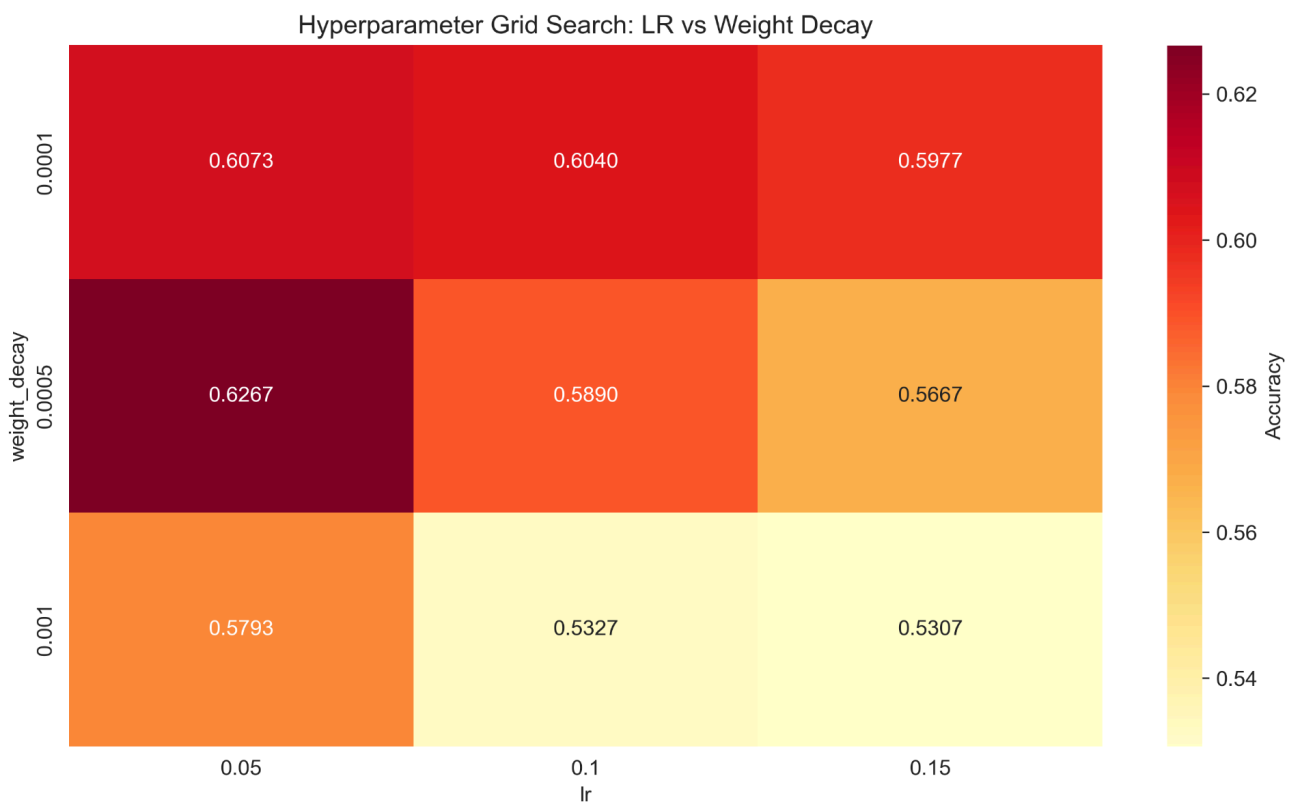


Figure 14: Hyperparameter Grid Search heatmap

This heatmap effectively identifies the optimal hyperparameter combination for the model, highlighting that a learning rate of 0.05 paired with a weight decay of 0.0005 yields the highest accuracy of 0.6267. This visual shows that while lower learning rates generally perform well, the interaction with weight decay is important, specifically, a moderate decay prevents overfitting better than higher values of 0.001 without hindering learning as much as lower values of 0.0001.

The validation loss and validation accuracy comparisons for different learning rates have been shown in the figure below:-

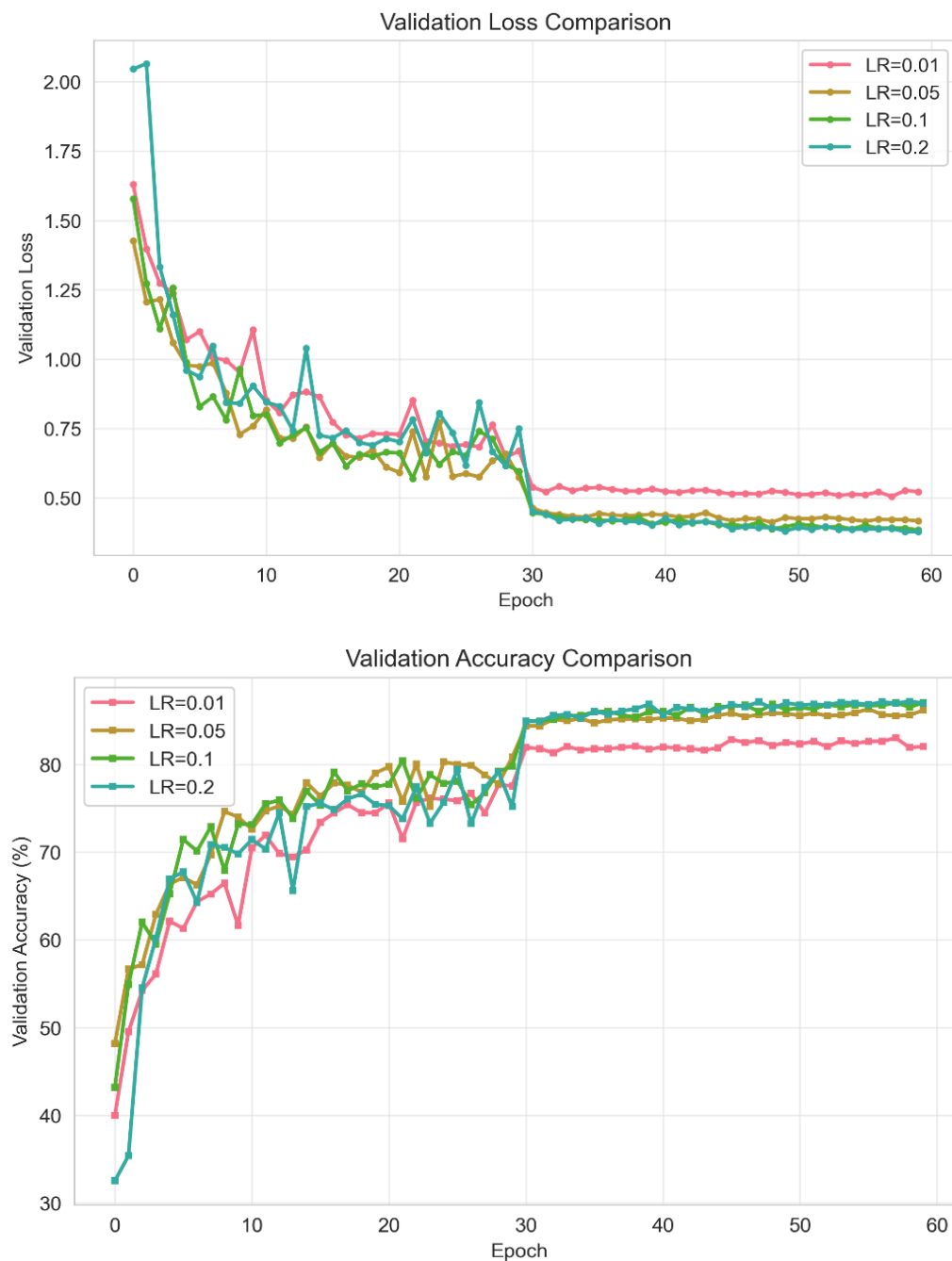


Figure 15: Validation loss and accuracy across different Learning Rates

These two charts illustrate the impact of learning rate on convergence speed and final performance over 60 epochs. The learning rate of 0.1 (green line) proves to be the most effective, demonstrating rapid convergence and high stability compared to the aggressive and unstable LR=0.2, while the lower rates of 0.01 and 0.05 converge too slowly to be competitive within the given timeframe.

The cross validation fold accuracy and accuracy distribution have been shown in the figure below:-

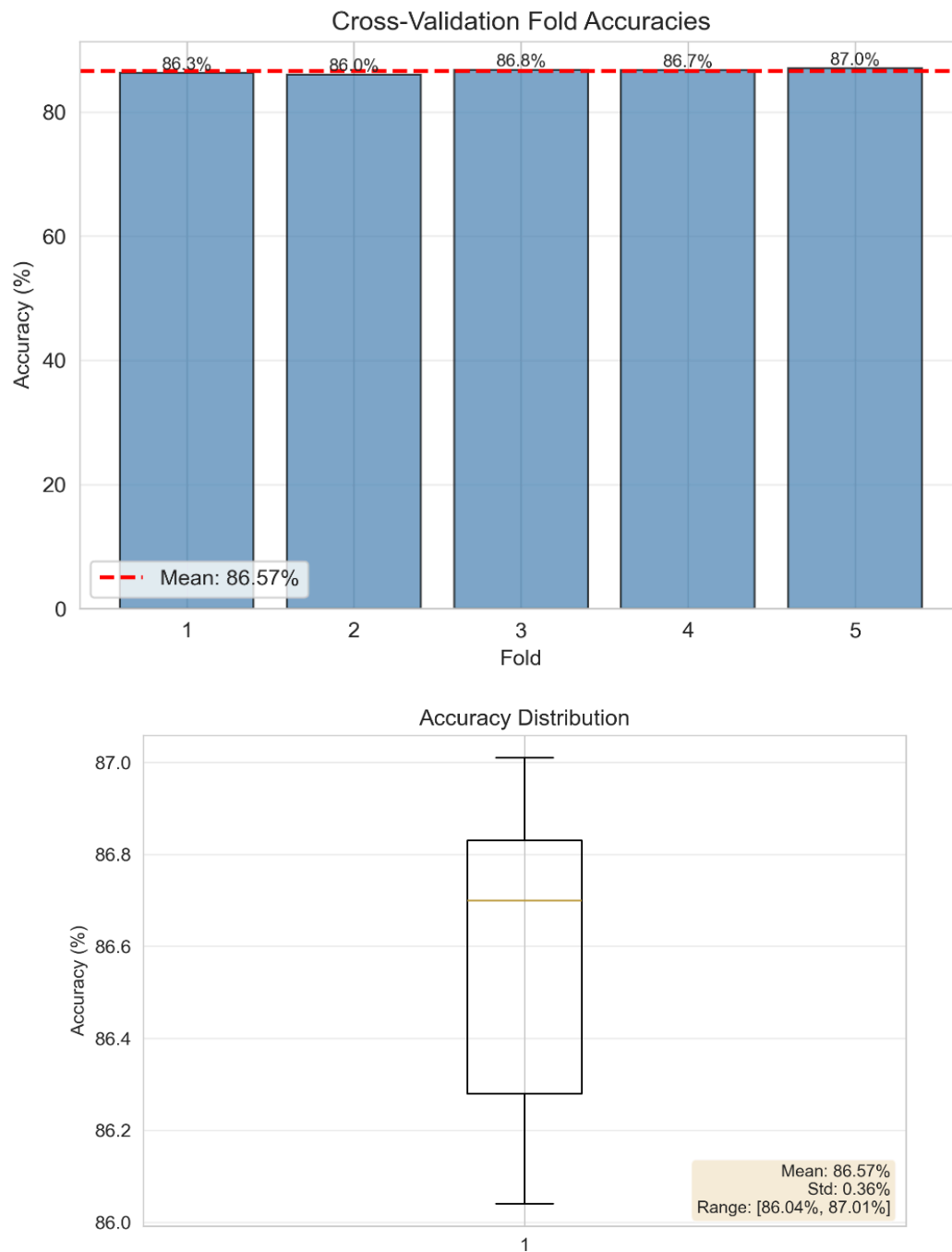


Figure 16: Cross Validation Fold accuracy and distribution

The charts above provide strong evidence for the model's robustness and generalizability. With a mean accuracy of 86.57% and an extremely low standard deviation of 0.36% across five different folds, the tight distribution in the box plot provides evidence that the model's performance is stable and not reliant on any specific data split, validating its readiness for deployment.

The bootstrap accuracy distribution and accuracy violin plot have been shown in the figure below:-

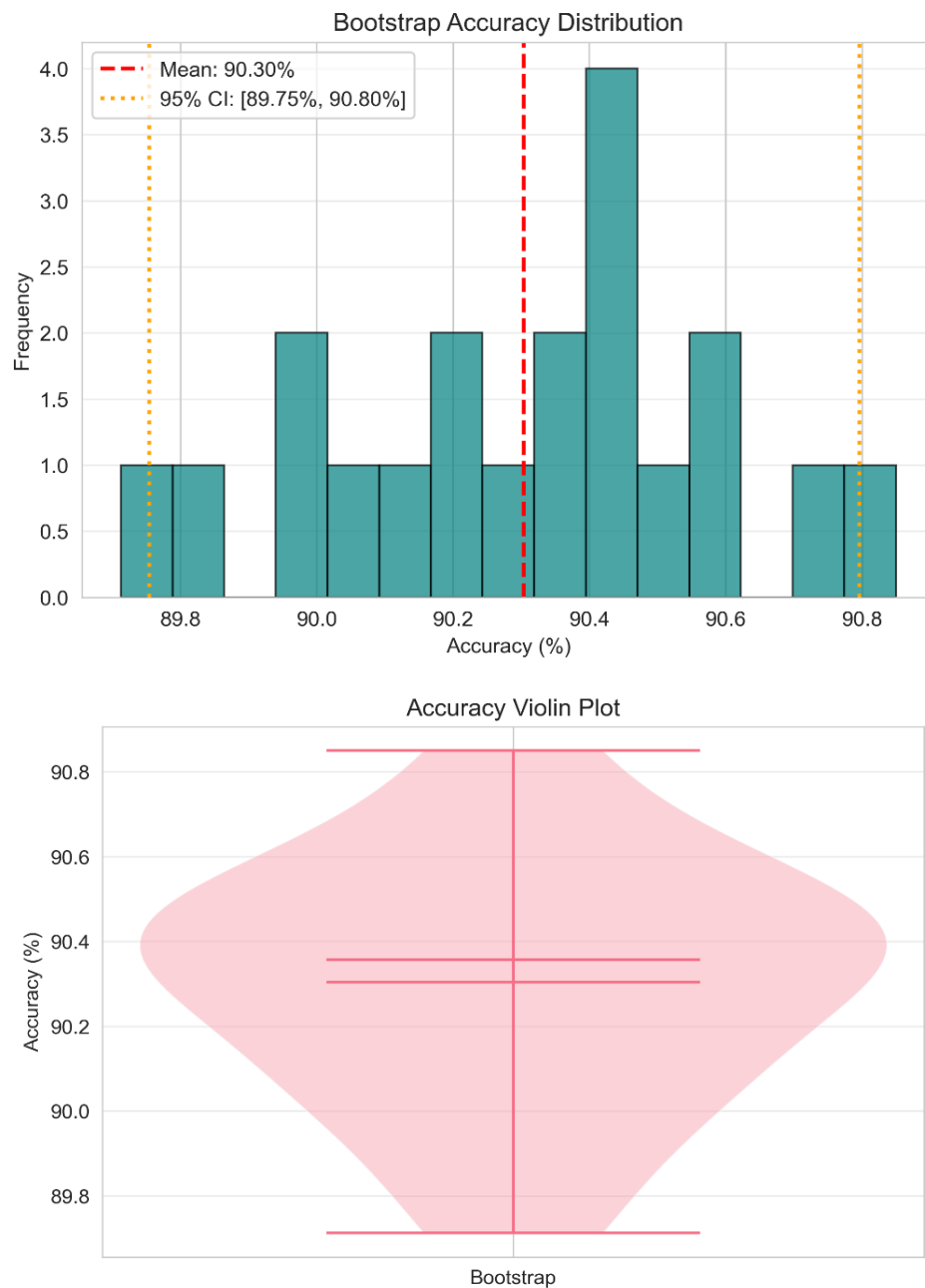


Figure 17: Bootstrap accuracy distribution and accuracy Violin Plot

The Bootstrap Accuracy Distribution bar chart and Accuracy Violin Plot show a statistical estimate of the model's expected performance on unseen data. The analysis reveals a mean accuracy of 90.30% with a 95% confidence interval, showing a high degree of certainty that the actual model performance lies within this high-accuracy range, further proving the effectiveness of the optimization efforts.

The loss and accuracy curves for the optimized ResNet-32 architecture are shown below:-

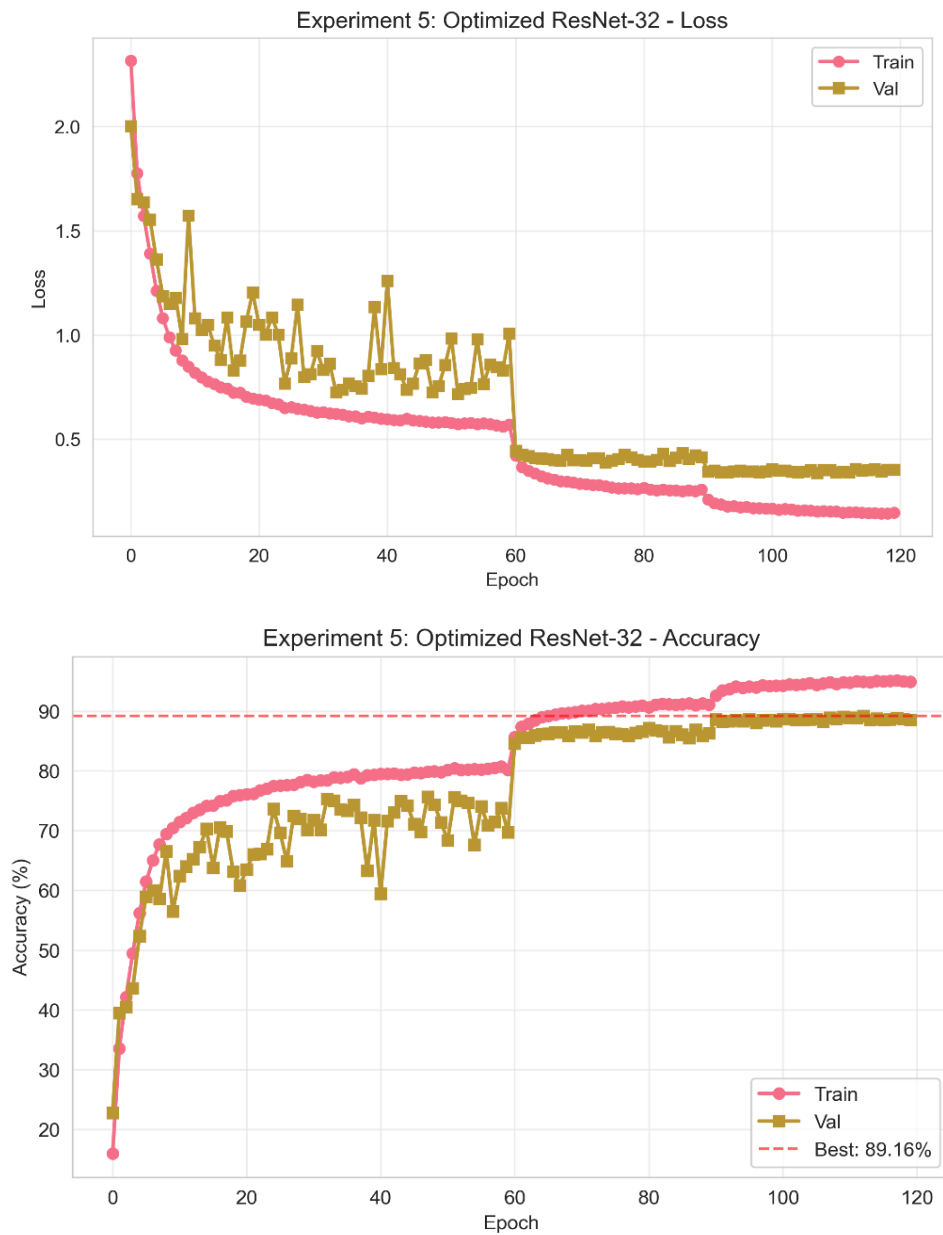


Figure 18: Loss and Accuracy compared across training and validation sets

For Experiment 5, the charts show the cumulative benefit of the optimization strategies applied. The training curves visualize a classic "stair-step" pattern, indicating the effective use of a learning rate scheduler. The points where improvements in accuracy and drops in loss occur correspond to where the learning rate has decayed. This allows the model to find a finer minima. The model reaches a peak validation accuracy of 89.16%, and the close tracking between training and validation curves suggests that the final model does not suffer from significant overfitting despite its increased complexity.

The test accuracy comparison for all the fundamental tasks have been shown in the figure below:-

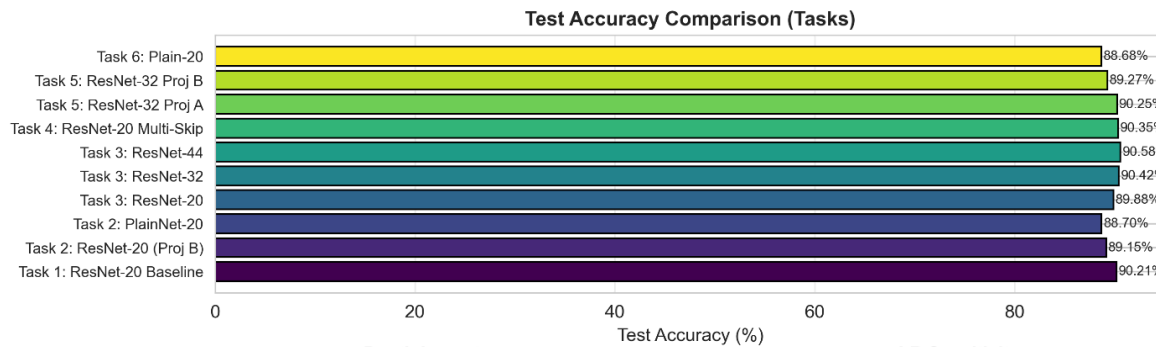


Figure 19: Test accuracy comparisons for all the tasks

This horizontal bar chart compares the test accuracy of various models across six tasks. The results show that deeper ResNet models generally perform better. Specifically, Task 3: ResNet-44 achieved the highest accuracy at 90.58%, followed closely by Task 3: ResNet-32 at 90.42% and Task 4: ResNet-20 Multi-Skip at 90.35%. In contrast, the shallower Task 6: Plain-20 model had the lowest accuracy at 88.68%. This suggests a positive correlation between network depth (and skip connections) and model performance on this dataset.

The validation accuracy of the top 3 task models are as follows:-



Figure 20: Validation accuracy comparison of top 3 task models

This line graph tracks the validation accuracy over 100 epochs for the top three performing models: ResNet-20, ResNet-20 (Projection B), and PlainNet-20. All three models show a rapid initial increase in accuracy, followed by fluctuations and a gradual plateau. The ResNet-20 appears to maintain the highest validation accuracy throughout the later epochs, confirming its superior stability and performance compared to the PlainNet-20, which exhibits more instability, mainly around epoch 40.

To understand the impact of network depth on the test accuracy, which is one of the major objectives of this report, the following line chart has been shown as follows:-

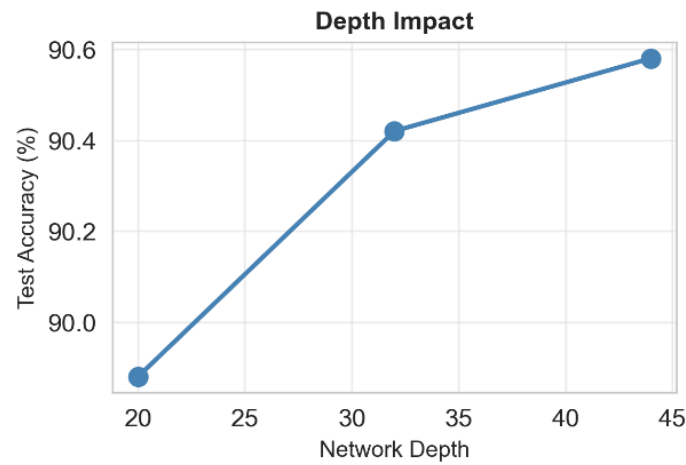


Figure 21: Impact of network depth on the test accuracy

This line chart illustrates the relationship between network depth and test accuracy. There is a clear positive trend: as the network depth increases from 20 to 32 and then to 44 layers, the test accuracy improves from roughly 89.9% to nearly 90.6%. This reinforces the finding from the first chart that deeper architectures are beneficial for this task. Furthermore, the impact of test accuracy due to the change in learning rate also needs to be studied, which has been visualised below:-

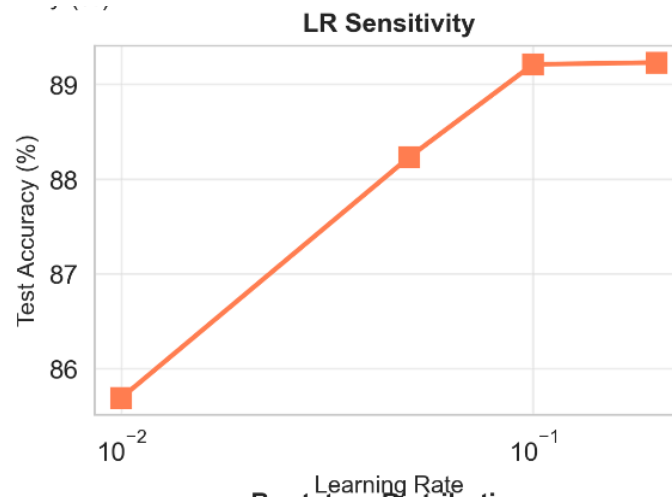


Figure 22: Learning rate impact on the test accuracy

This line chart plots test accuracy against the learning rate. It shows a positive linear relationship between the learning rate and model accuracy within the tested range (0.01 to 0.1). Higher learning rates closer to 0.1 yielded significantly higher test accuracy than lower rates, indicating that the model benefits from a more aggressive learning schedule.

The test accuracy comparisons for two different projection methods have been shown below:-

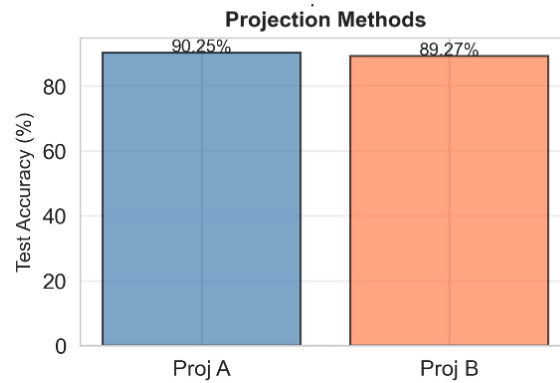


Figure 23: Comparison of test accuracy for different projection methods

This bar chart compares two projection shortcut methods, Projection A i.e. the zero-padding shortcut and Projection B i.e. the convolutional shortcut. Projection A achieved a higher test accuracy of 90.25% compared to Projection B's 89.27%. This suggests that zero-padding shortcuts might be more effective or at least sufficient for this specific ResNet architecture compared to projection shortcuts (Option B) which introduce extra parameters.

The test accuracy across different skip connection types have been visualized below:-

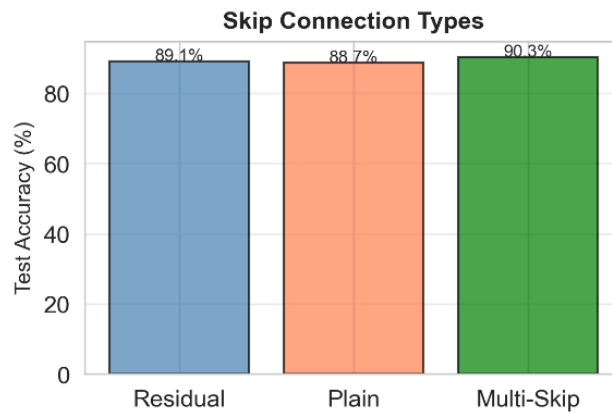


Figure 24: Comparison of test accuracy for different skip connection types

This bar chart compares the test accuracy of three architectural variants: Residual, Plain, and Multi-Skip. The Multi-Skip connection type achieved the highest accuracy at 90.3%, followed by Residual at 89.1%, and Plain at 88.7%. This highlights the effectiveness of skip connections, particularly multi-skip variations, in improving model performance over plain architectures without such connections.

The bootstrap distribution's accuracy across different frequency has been shown below:-

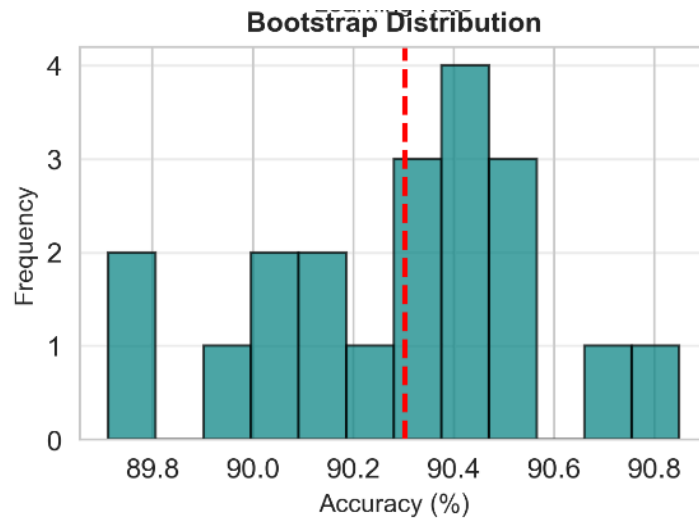


Figure 25: Bootstrap distribution showing accuracy across frequencies

This histogram shows the frequency distribution of accuracy scores obtained from bootstrap resampling. The distribution is roughly centered around a mean accuracy of approximately 90.3% (indicated by the red dashed line), with scores ranging from roughly 89.8% to 90.8%. This provides an estimate of the model's performance variability and confidence interval.

The accuracy across different number of folds while using cross validation has been shown below:-

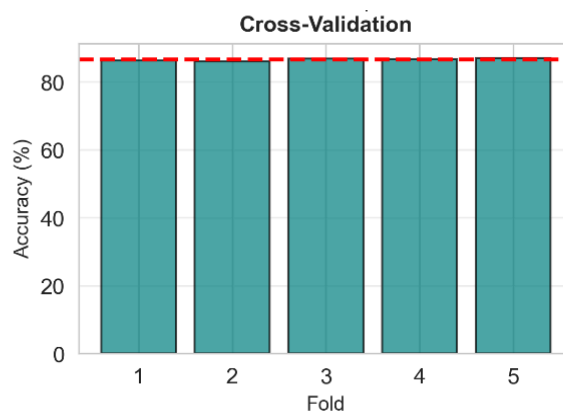


Figure 26: Cross validation for number of folds and corresponding accuracy

This bar chart displays the accuracy across 5 folds of cross-validation. The accuracy is consistent across all five folds, hovering around the mean (indicated by the red dashed line), which appears to be approximately 87-88%. This consistency suggests that the model's performance is robust and not dependent on a specific data split.

The results of tasks performed in the project have been summarized in the table below:-

Task	Validation Accuracy	Test Accuracy	Test Loss
Task 1: ResNet-20 Baseline	88.42%	90.21%	0.3161
Task 2: ResNet-20 (Convolutional shortcut)	87.54%	89.15%	0.3345
Task 2: PlainNet-20	86.20%	88.70%	0.3473
Task 3: ResNet-20	87.41%	89.88%	0.3293
Task 3: ResNet-32	88.14%	90.42%	0.3223
Task 3: ResNet-44	89.06%	90.58%	0.3077
Task 4: ResNet-20 Multi-Skip	88.90%	90.35%	0.3144
Task 5: ResNet-32 (Zero-padding shortcut)	88.63%	90.25%	0.3136
Task 5: ResNet-32 (Convolutional shortcut)	86.92%	89.27%	0.3446
Task 6: Plain-20	86.29%	88.68%	0.2652
Task 6: Plain-32	84.72%	87.65%	0.3322
Task 6: Plain-44	81.38%	84.48%	0.4136
Task 6: Plain-56	79.61%	82.67%	0.5058
Task 6: ResNet-20	86.51%	89.06%	0.2502
Task 6: ResNet-32	87.17%	88.74%	0.2298
Task 6: ResNet-44	86.86%	89.00%	0.2049
Task 6: ResNet-56	88.84%	90.65%	0.1564

Table 4: Summarized results of different tasks

This table shows the overall results of the tasks performed. Initial grid searches established a baseline optimal learning rate of 0.05 and weight decay of 0.0001, which were important for later steps. The learning rate sensitivity analysis in Experiment 2 showed a clear positive correlation between higher learning rates and accuracy, with rates of 0.1 and 0.2 pushing validation accuracy to approximately 87%. The cross-validation and bootstrap analysis in Experiments 3 and 4 demonstrated the model's stability and narrowed the expected performance range to a 95% confidence interval of 89.75% to 90.80%. This process showed results in Experiment 5, where the fully optimized model achieved a peak test accuracy of 91.16% and the lowest observed test loss of 0.2943.

The results of experiments performed in the project have been summarized in the table below:-

Experiment	Best Learning Rate (LR)	Best Weight Decay (WD)	Best Momentum	Best Validation Accuracy
Grid Search	0.05	0.0001	0.9	63.50%

Experiment	Val Acc	Test Acc	Test Loss
Experiment 2: LR = 0.01	83.01%	85.69%	0.4331
Experiment 2: LR = 0.05	86.32%	88.23%	0.3587
Experiment 2: LR = 0.1	87.03%	89.21%	0.3364
Experiment 2: LR = 0.2	87.19%	89.23%	0.3335
Experiment 5: Optimized	89.16%	91.16%	0.2943

Experiment	Mean Acc	Standard Deviation	95% CI
5 - Fold Cross Validation	86.57%	0.36%	[86.04%, 87.01%]
Bootstrap	90.30%	0.29%	[89.75%, 90.80%]

Table 5: Summarized results of different experiments

This table shows performance tiers across different model architectures. Deep Convolutional Neural Networks (CNNs) consistently outperformed the shallower models. The ResNet-44 model achieved the highest test accuracy of 90.58% which surpassed both the ResNet-32 (90.42%) and the ResNet-20 baseline (90.21%). The data also highlighted the limitations of depth without residual connections i.e. the 56-layer PlainNet faced significant degradation, dropping to 82.67% accuracy compared to the 88.68% of the 20-layer version. Advanced architectural modifications proved to be highly effective, with the Multi-Skip ResNet-20 variant achieving 90.35% accuracy. This suggests that optimizing connection paths can yield performance gains comparable to simply adding more layers.

Discussion

Performance Analysis of Residual Architecture and Plain Architecture

The results provided evidence preferring the residual learning architectures over traditional plain networks, particularly as network depth increased. The baseline ResNet-20 achieved 90.21% test accuracy, establishing a strong foundation that was consistently matched or exceeded by deeper residual variants. The ResNet-44 reached 90.58% accuracy, demonstrating that the addition of residual connections enabled effective training of deeper networks. This finding aligned with theoretical predictions regarding the ability of skip connections to facilitate gradient flow through deep architectures.

On the other hand, plain networks exhibited severe performance degradation as depth increased. The PlainNet-20 achieved 88.68% test accuracy, already lagging behind its residual counterpart by 1.53 percentage points. More significantly, as depth increased to 56 layers, plain network performance collapsed to 82.67%, representing a 6.01 percentage point decline from the 20-layer baseline. This degradation occurred despite all networks being trained with identical optimization protocols, clearly indicating that the performance decline was not attributable to overfitting but rather to fundamental optimization difficulties in deep plain architectures.

The divergent behavior between residual and plain networks became particularly evident when examining training dynamics. Plain networks showed progressively higher training loss as depth increased, with the 56-layer variant reaching 0.5058 compared to 0.2652 for the 20-layer version. This pattern suggested that deeper plain networks struggled to optimize even the training set, confirming that the degradation problem stemmed from optimization challenges rather than generalization failure. Residual networks, conversely, achieved progressively lower training losses with increased depth, with ResNet-56 attaining 0.1564, demonstrating that skip connections successfully addressed the optimization pathology afflicting plain networks.

Architecture Selection

The investigation of projection methods for matching dimensions in skip connections revealed some performance differences. ResNet-32 with option A projection (zero-padding) achieved 90.25% test accuracy, outperforming option B (1×1 convolution projection) which reached 89.27%. This 0.98 percentage point difference suggested that the zero-padding approach, despite introducing no additional parameters, provided sufficient representational capacity for effective residual learning. The computational efficiency of option A, combined with its competitive performance, made it an attractive choice for resource-constrained scenarios. Multi-layer skip connections that bypassed three convolutional layers instead of the standard two demonstrated promising results, with the modified ResNet-20 achieving 90.35% test accuracy compared to 90.21% for the standard configuration. This modest improvement of 0.14 percentage points suggested that extending skip paths could provide incremental benefits, though the magnitude of improvement did not justify the added architectural complexity for most applications. The result indicated that the standard two-layer skip configuration represented a well-optimized design point balancing performance and simplicity.

The relationship between network depth and performance in residual architectures proved more complex than simple monotonic improvement. While ResNet-44 achieved the highest test accuracy among the depth comparison experiments at 90.58%, ResNet-32 with optimized training configuration reached 91.16%, surpassing all other variants. This outcome highlighted the important relation between architectural depth and training methodology, suggesting that deeper networks required more carefully tuned hyperparameters to realize their full potential.

Challenges

Some of the challenges that were encountered during the project are as follows:-

1. The normalization process failed multiple times at the initial phase when the data was loaded as 4-dimensional tensors were expected but 3-dimensional single images were only provided, which required modifying both cases and also included conditional dimension checking.
2. The training part in total required 450 epochs and the session was continuously being crashed in different platforms due to minor issues with the code or with the system. The first successful and complete execution of code was recorded only after 8 failed attempts that had wasted several days.
3. The process of debugging the entire code after completing more than half of the planned tasks started becoming very tedious and difficult as the requirements and expectations of different functions started diverging which raised issues alternatively in different functions.

Limitations

Some of the limitations of the project that were identified are as follows:-

1. It focused only on CIFAR-10, a relatively small-scale dataset with low-resolution 32x32 images, limiting generalization insights to larger datasets like ImageNet or higher-resolution images, though the fundamental optimization advantages of residual connections likely transfer across scales.
2. The computational constraints of our devices limited the deepest networks to 56 layers, leaving unexplored the behavior of very deep residual networks exceeding 100 layers that the original ResNet paper demonstrated could scale successfully, and training duration of 80 to 100 epochs represented a shortcoming that limited convergence for deeper networks.

Future Enhancements

Some possible future enhancements that had not been implemented for present due to time constraints but can be implemented in the future are as follows:-

1. Implement deeper ResNet variants including ResNet-101 and ResNet-152 with bottleneck blocks on larger datasets such as ImageNet or CIFAR-100 to analyze scalability and compare computational efficiency against modern architectures like EfficientNet and Vision Transformers.
2. Extend the framework to incorporate advanced residual variants including ResNeXt with grouped convolutions, Squeeze-and-Excitation blocks for channel attention, and pre-activation residual units to evaluate their impact on accuracy and training dynamics.
3. Integrate transfer learning experiments, mixed-precision training for computational efficiency, and proper research on advanced data augmentation strategies to optimize model performance.

Conclusion

This project demonstrated the advantages of residual learning architectures using the CIFAR-10 dataset. It confirmed the degradation problem in deep plain networks, with a 56-layer plain network underperforming its 20-layer counterpart by 6.01 percentage points, accompanied by increasing training loss, indicating optimization difficulties rather than overfitting. Residual networks resolved this issue, with a 56-layer ResNet achieving 90.65% test accuracy versus 89.06% for 20 layers, showing that skip connections enable effective training of deeper networks. Architectural comparisons revealed that Option A projection (zero-padding) outperformed Option B (1×1 convolution) while requiring no extra parameters, favoring simpler identity mappings. Multi-layer skip connections offered modest improvements of 0.14 percentage points, suggesting limited benefit. The optimized ResNet-32 reached 91.16% test accuracy, highlighting that careful hyperparameter tuning enhances architectural advantages. From a practical perspective, ResNet-32 at 0.46 million parameters achieved optimal performance-efficiency balance, while lighter ResNet-20 at 0.27 million parameters and 90.21% accuracy offered a resource-friendly alternative.

This implementation successfully validates the core principle that skip connections enable stable training of deep networks by reproducing key findings from the original ResNet paper. The project demonstrates how residual learning resolves the degradation problem through systematic experiments on CIFAR-10, showing that plain networks fail to optimize effectively at greater depths while residual networks scale successfully. By implementing and comparing networks at depths of 20, 32, 44, and 56 layers, the code confirms that residual connections create gradient highways allowing direct backpropagation paths, which enables the training of deeper architectures. The experiments including architectural variants, hyperparameter optimization, and statistical validation provide practical insights into the design principles that have become foundational across modern deep learning architectures including ResNeXt, DenseNet, transformers, and efficient mobile networks, demonstrating the universal applicability of residual learning for training deep neural networks.

References

- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).
- Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2), 157-166.
- Glorot, X., & Bengio, Y. (2010, March). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (pp. 249-256). JMLR Workshop and Conference Proceedings.
- Srivastava, R. K., Greff, K., & Schmidhuber, J. (2015). Training very deep networks. *Advances in neural information processing systems*, 28.
- Vatanen, T., Raiko, T., Valpola, H., & LeCun, Y. (2013, November). Pushing stochastic gradient towards second-order methods—backpropagation learning with transformations in nonlinearities. In *International conference on neural information processing* (pp. 442-449). Berlin, Heidelberg: Springer Berlin Heidelberg.
- Pimentel, T., Valvoda, J., Stoeck, N., & Cotterell, R. (2022). The architectural bottleneck principle. *arXiv preprint arXiv:2211.06420*.
- M, S. (2025, February 24). *Understanding resnet and analyzing various models on the CIFAR-10 dataset*. Analytics Vidhya.
<https://www.analyticsvidhya.com/blog/2021/06/understanding-resnet-and-analyzing-various-models-on-the-cifar-10-dataset/>
- VGG-19 convolutional neural network - an overview | sciencedirect topics. (n.d.).
<https://www.sciencedirect.com/topics/computer-science/vgg-19-convolutional-neural-network>

Appendices

The fundamental slides for the project presentation has been shown below:-

Introduction

- Deep networks become hard to train when layers increase.
- Accuracy gets worse instead of better (degradation problem).
- ResNet introduces skip connections to solve this.
- Helps train very deep models easily.
- Learns residuals: $\text{output} = \text{input} + \text{small change } (F(x) + x)$

3



Objectives

- Make deep neural networks easier to train and optimize
- Overcome the degradation problem where accuracy drops as layers increase
- Reduce training error even in very deep networks
- Successfully train extremely deep architectures (50–152 layers on ImageNet)

4



Related work

- **Residual Representations**
 - Effective across different fields, such as image recognition (VLAD, Fisher Vector) and solving PDEs (Multigrid method), because expressing solutions or features as residuals simplifies the underlying optimization problem and leads to faster convergence or more effective encoding.
- **Shortcut Connections**
 - These connections can add linear layers to MLPs and connect intermediate layers to auxiliary classifiers. While related work like "highway networks" use gated shortcuts and can be closed, the ResNet approach uses parameter-free identity shortcuts that are never closed, ensuring information always passes through and the formulation consistently learns residual functions.

Methodology

- Instead of learning $H(x)$, each block learns a **residual function**, $F(x) = H(x) - x$ such that $H(x)$ can be obtained as $F(x) + x$.
- Identity **shortcut connections** are used to add input x directly to output $F(x)$, which allows gradients to flow across layers directly.
- If residuals tend to be zero, identity mapping is performed so that deeper models yield results as good as other models.
- Residual blocks are **stacked hierarchically** for depth scalability.
- Projection shortcuts are applied to match the feature dimensions between layers.
- Feature reuse becomes possible improving the representational richness.

Methodology

1. Each block learnt a **residual function** $F(x) = H(x) - x$, producing output $H(x) = F(x) + x$.
2. **Identity shortcuts** added x directly to $F(x)$, improving gradient flow.
3. The block performed **identity mapping** when $F(x) = 0$, preventing degradation with depth.
4. **Residual blocks** were **hierarchically stacked** to scale the depth.
5. Projection shortcuts (1×1 conv) were aligned to feature dimensions.
6. Skip connections enabled **feature reuse**, improving generalization and representations.

Methodology

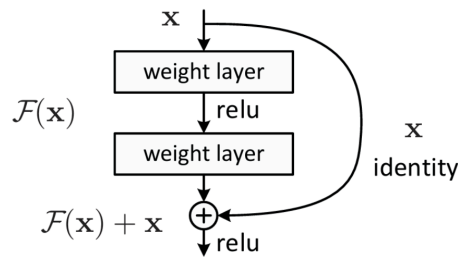


Figure 2. Residual learning: a building block.

Figure adapted from:
He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Deep Residual Learning for Image Recognition*.
arXiv:1512.03385.

Results

- **Degradation problem is solved** i.e., deeper networks trained better.
- 34 layer ResNet **surpassed VGG-19** with fewer operations.
- ResNet - 50, 101, and 152 provided **progressively lower** classification errors.
- 152-layer ResNet achieved 4.49% top-5 validation error on ImageNet
- Improved object detection and localization tasks on the COCO dataset.
- Even at greater depths, required **less computation** and **fewer parameters** as compared to plain deep networks.

Conclusions

- Very deep networks can be effectively trained using Residual learning.
- Shortcut connections ensure stable gradient flow and easier optimization.
- Elimination of **degradation problem**, such that deeper models perform as good as or better than shallower models.
- Same framework can be used to improve detection, segmentation, and recognition.
- Foundation for several modern architectures like **ResNeXt**, **DenseNet**, **Transformers** etc.
- Breakthrough in deep learning with combination of **accuracy**, **depth**, and **computational efficiency**.

References

- He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Deep Residual Learning for Image Recognition*. arXiv:1512.03385. (CVPR 2016)
- Bengio et al. (1994) — *Learning long-term dependencies with gradient descent is difficult* (IEEE TNN)
- Glorot & Bengio (2010) — *Understanding the difficulty of training deep feedforward neural networks* (AISTATS)
- Srivastava et al. (2015) — *Training very deep networks* (arXiv:1507.06228)

11



Figure 27: Fundamental presentation slides

The formats of the actual outputs of the code have been cropped in case of images, and shown in actual texts instead of images for table results. The actual results yielded from the code are as follows:-

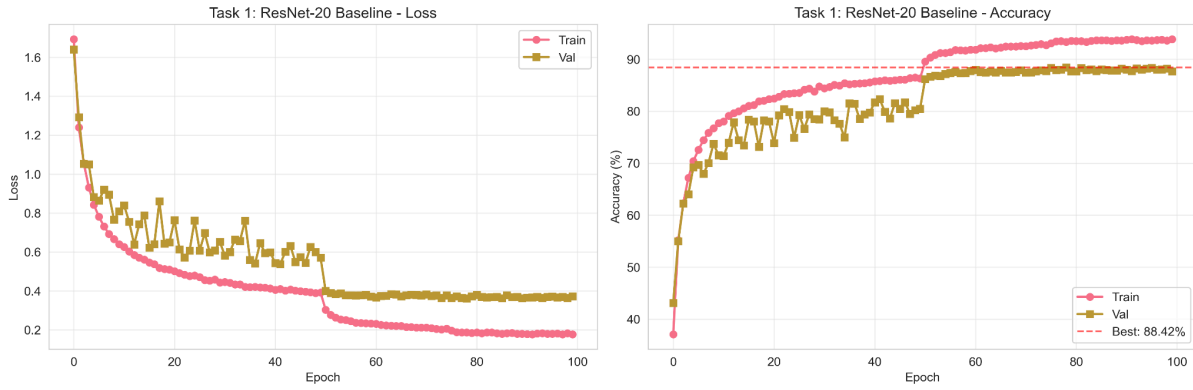


Figure 28: Output visualization for Task 1

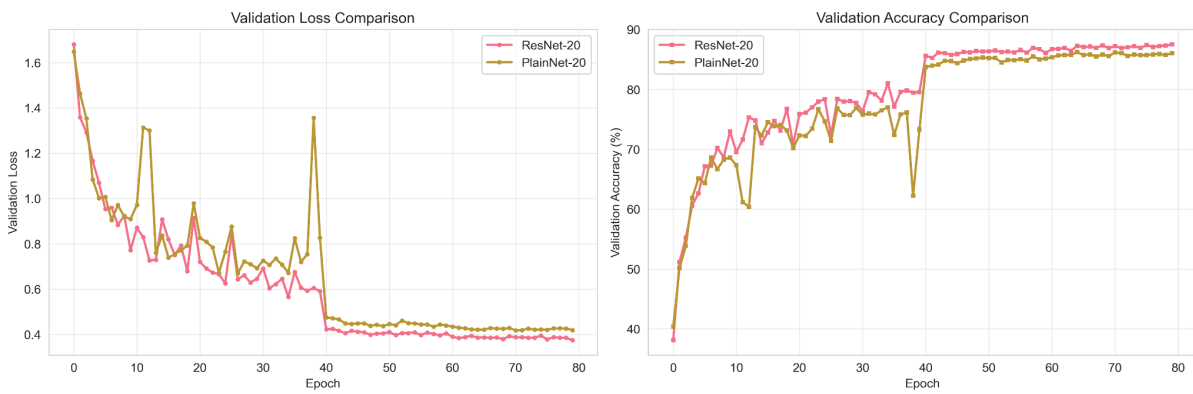


Figure 29: Output visualization for Task 2

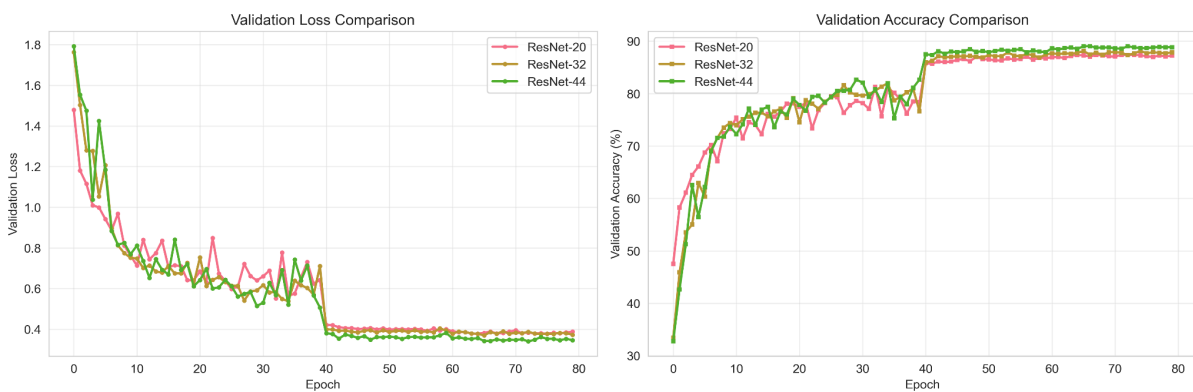


Figure 30: Output visualization for Task 3

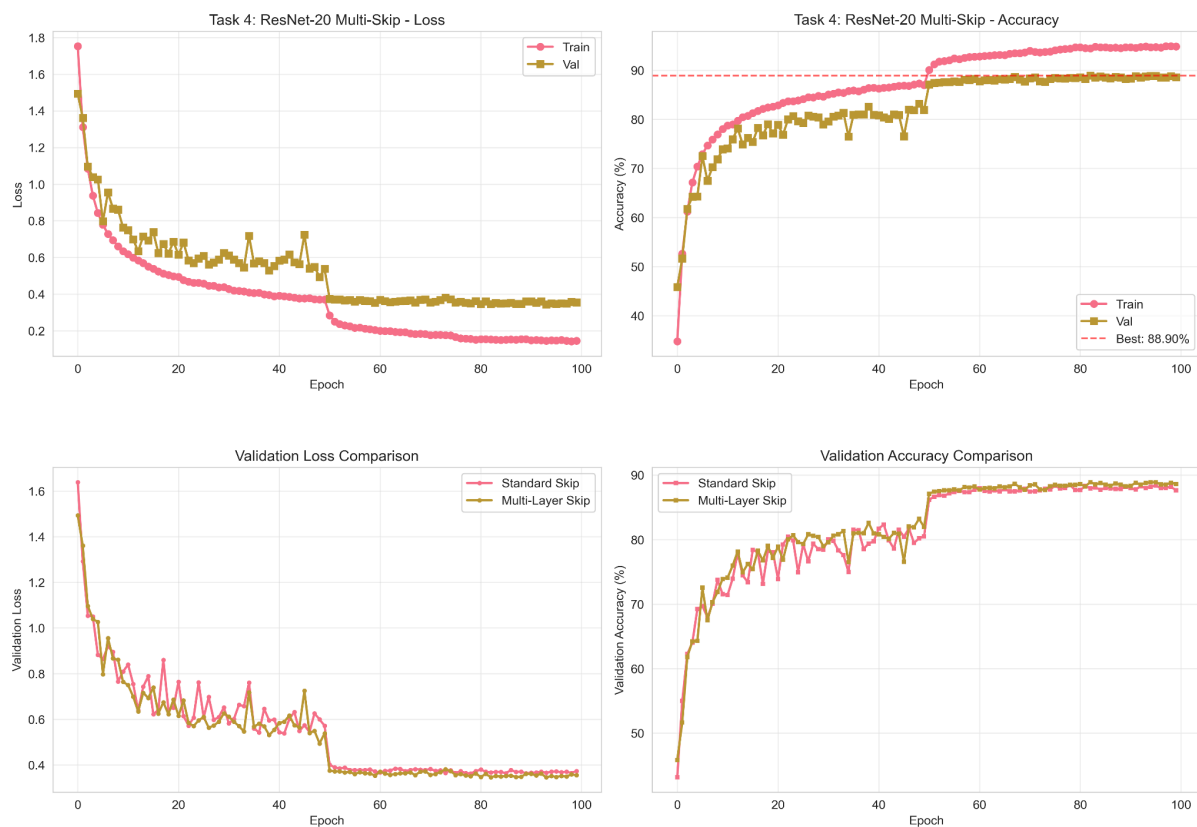


Figure 31: Output visualization for Task 4

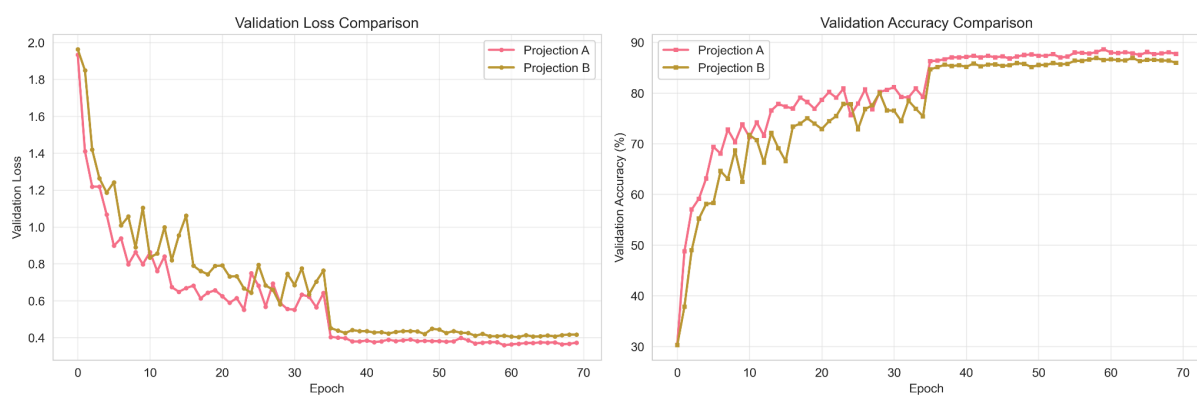


Figure 32: Output visualization for Task 5

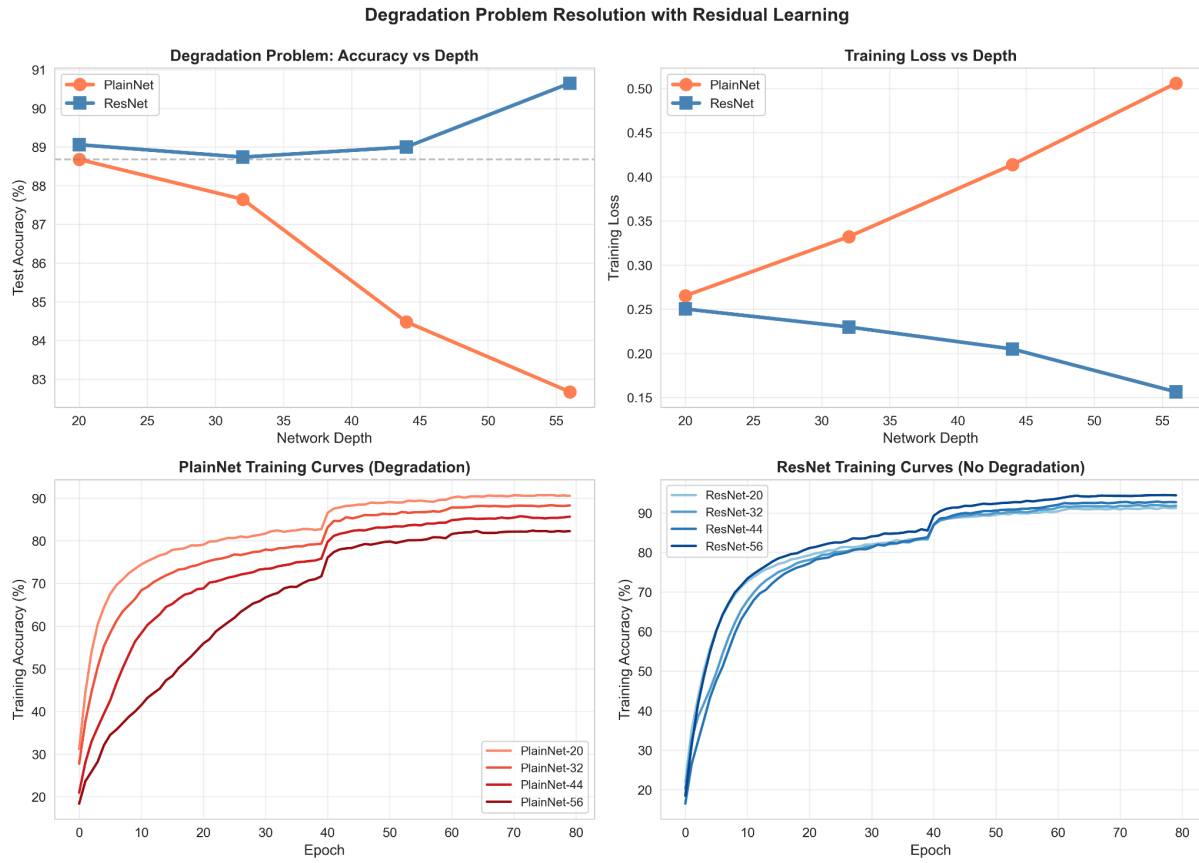


Figure 33: Output visualization for Task 6

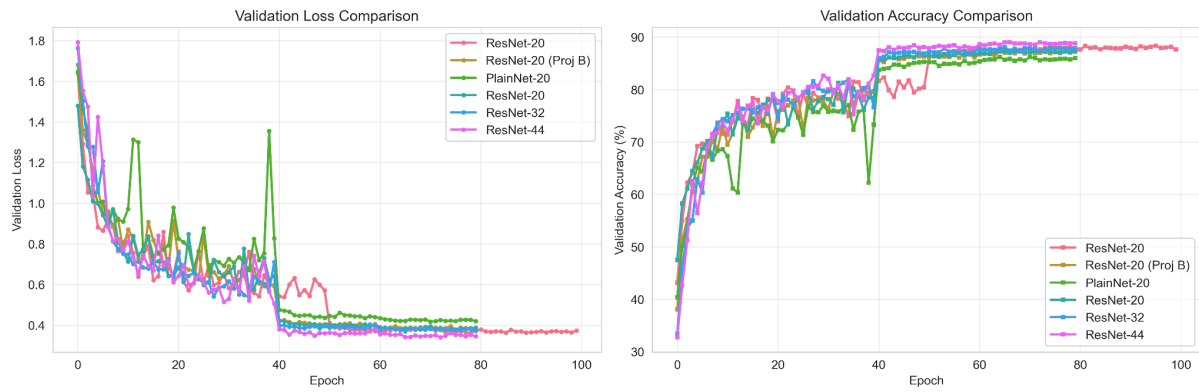


Figure 34: Output visualization for summary of all the tasks

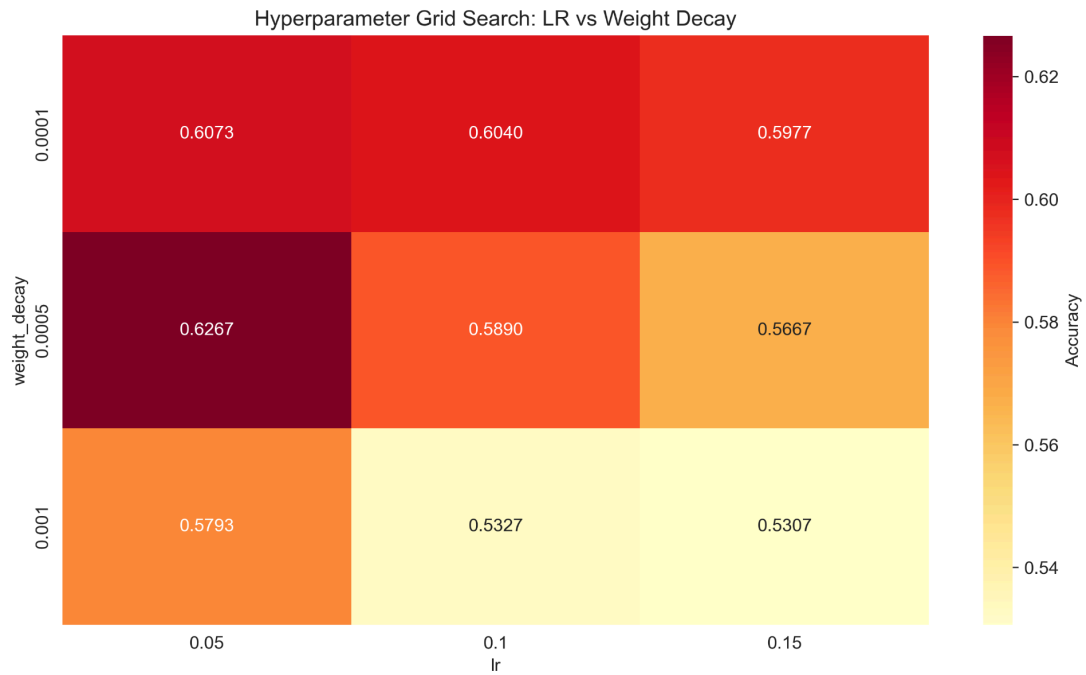


Figure 35: Output visualization for Experiment 1

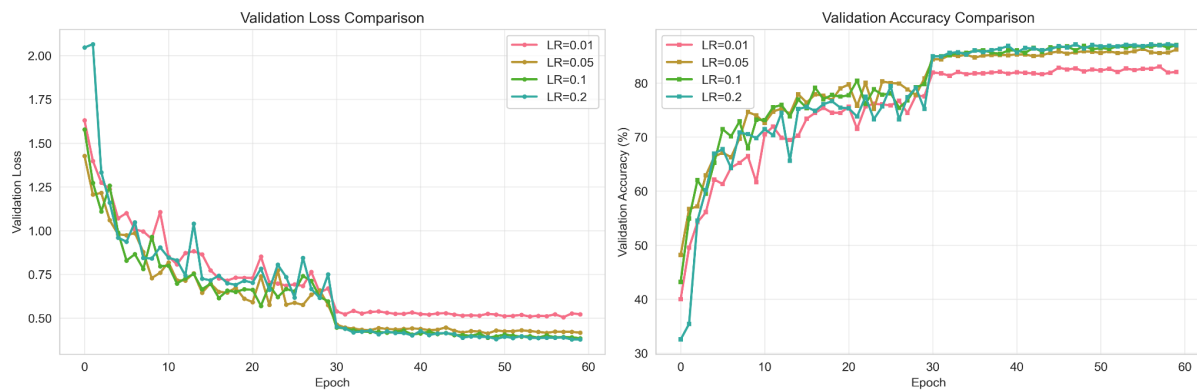


Figure 36: Output visualization for Experiment 2

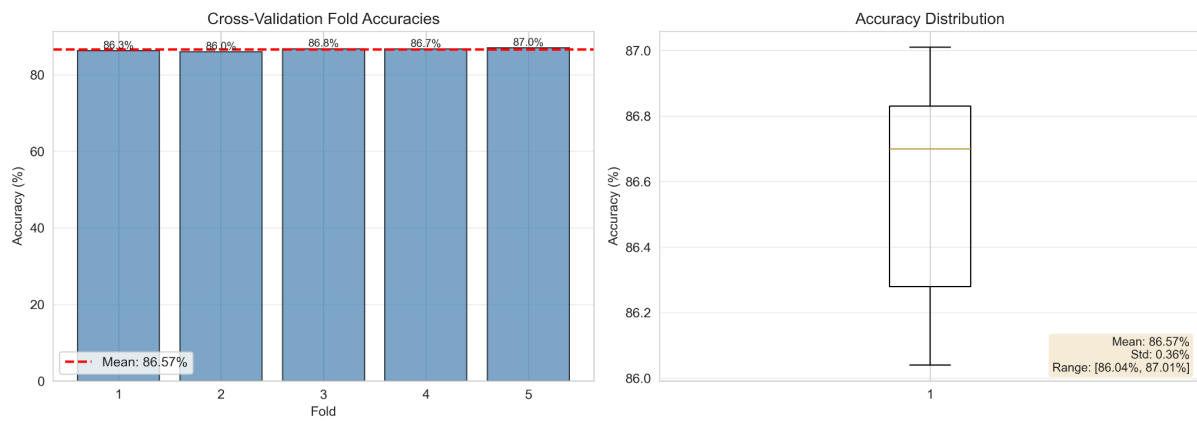


Figure 37: Output visualization for Experiment 3

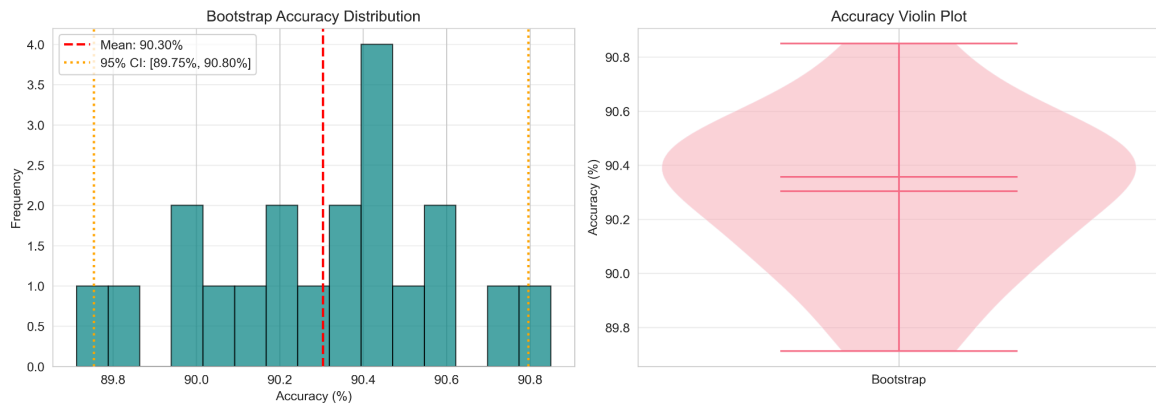


Figure 38: Output visualization for Experiment 4

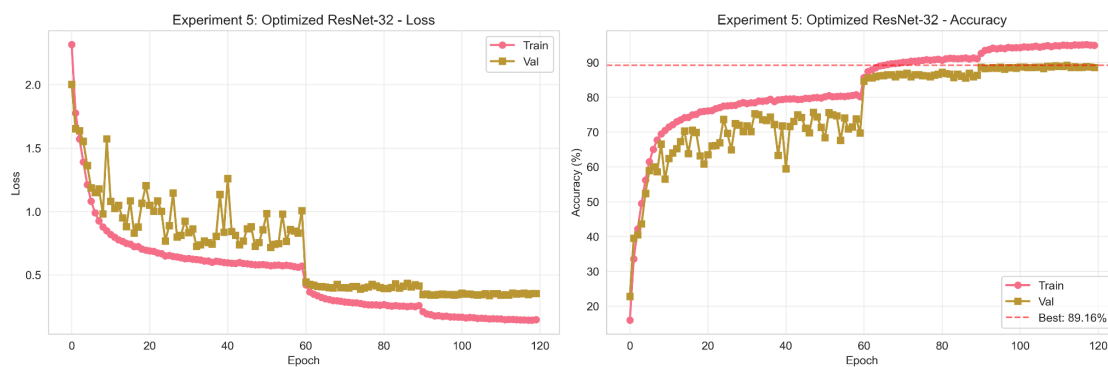


Figure 39: Output visualization for Experiment 5

Comprehensive Results Summary

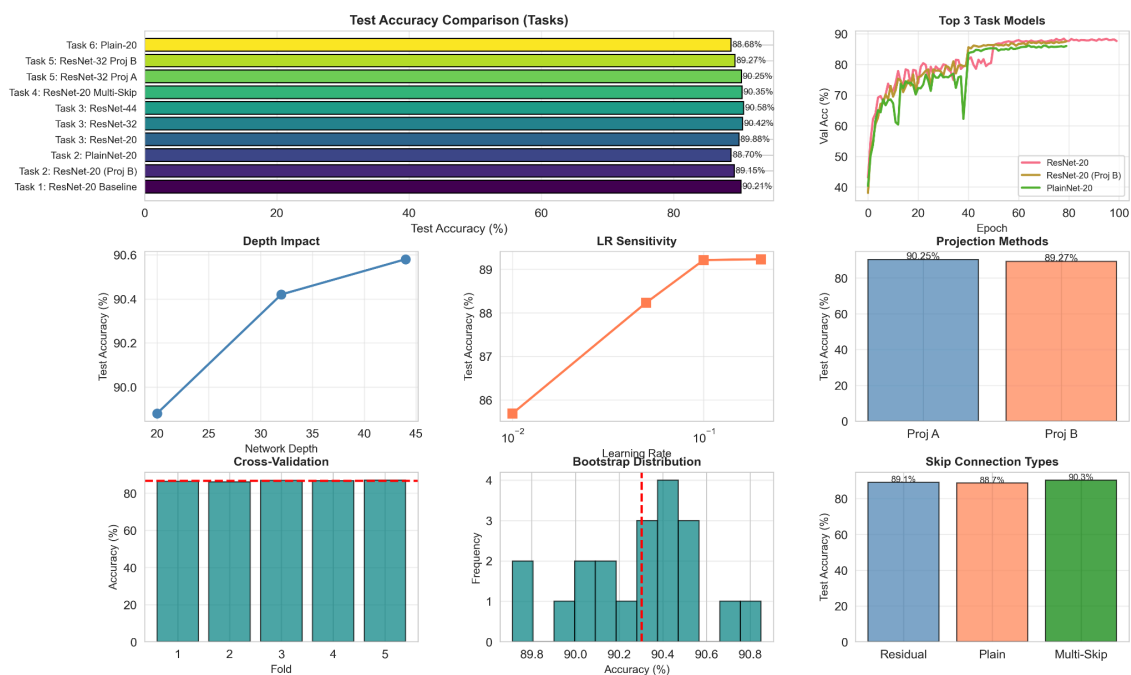


Figure 40: Output visualization for summary of results

Experiment Results Summary

Task	Val_Acc	Test_Acc	Test_Loss
Task 1: ResNet-20 Baseline	88.42%	90.21%	0.3161
Task 2: ResNet-20 (Proj B)	87.54%	89.15%	0.3345
Task 2: PlainNet-20	86.20%	88.70%	0.3473
Task 3: ResNet-20	87.41%	89.88%	0.3293
Task 3: ResNet-32	88.14%	90.42%	0.3223
Task 3: ResNet-44	89.06%	90.58%	0.3077
Task 4: ResNet-20 Multi-Skip	88.90%	90.35%	0.3144
Task 5: ResNet-32 Proj A	88.63%	90.25%	0.3136
Task 5: ResNet-32 Proj B	86.92%	89.27%	0.3446
Task 6: Plain-20	86.29%	88.68%	0.2652
Task 6: Plain-32	84.72%	87.65%	0.3322
Task 6: Plain-44	81.38%	84.48%	0.4136
Task 6: Plain-56	79.61%	82.67%	0.5058
Task 6: ResNet-20	86.51%	89.06%	0.2502
Task 6: ResNet-32	87.17%	88.74%	0.2298
Task 6: ResNet-44	86.86%	89.00%	0.2049
Task 6: ResNet-56	88.84%	90.65%	0.1564

Figure 41: Output table to summarize results of all tasks

Experiment Results Summary

Experiment	Best_LR	Best_WD	Best_Momentum	Best_Val_Acc	Val_Acc	Test_Acc	Test_Loss	Mean_Acc	Std	CI_Range	CI_95
Experiment 1: Grid Search	0.05	0.0001	0.9	63.50%	nan	nan	nan	nan	nan	nan	nan
Experiment 2: LR=0.05	nan	nan	nan	nan	83.01%	85.69%	0.4331	nan	nan	nan	nan
Experiment 2: LR=0.01	nan	nan	nan	nan	86.32%	88.23%	0.3587	nan	nan	nan	nan
Experiment 2: LR=0.001	nan	nan	nan	nan	87.03%	89.21%	0.3364	nan	nan	nan	nan
Experiment 2: LR=0.0001	nan	nan	nan	nan	87.19%	89.23%	0.3335	nan	nan	nan	nan
Experiment 3: 5-Fold Cross Validation	nan	nan	nan	nan	nan	nan	nan	86.57%	0.36%	[86.04%, 87.01%]	nan
Experiment 4: Bootstrapping	nan	nan	nan	nan	nan	nan	nan	90.30%	0.29%	nan	[89.75%, 90.80%]
Experiment 5: Optimized	nan	nan	nan	nan	89.16%	91.16%	0.2943	nan	nan	nan	nan

Figure 42: Output table to summarize results of all experiments

Figures from Figure 6 onward were generated directly from the code, and some of these figures were cropped to improve presentation quality. Figures 1 through 5 were created using draw.io.